

BIOINFORMATICS TUTORIAL FOR THE ANALYSIS OF GENOME SEQUENCES OF GAMMA IRRADIATED BANANAS AND BENEFICIAL MICROBES

**The Joint FAO/IAEA Centre of Nuclear Techniques in Food and
Agriculture**

Vienna, 2026

DISCLAIMER

The mention of specific companies or products of specific manufacturers in this document does not imply that the Joint FAO/IAEA Centre endorses or recommends them in preference to others of a similar nature that are not mentioned.

The correct way to cite this document is:

The Joint FAO/IAEA Centre. 2026. BIOINFORMATICS TUTORIAL FOR THE ANALYSIS OF GENOME SEQUENCES OF GAMMA IRRADIATED BANANAS AND BENEFICIAL MICROBES.. Ames Mercedes, Qureshi Naeela and Cinthya Zorrilla (eds.), Food and Agriculture Organization /International Atomic Energy Agency. Vienna, Austria.

FOREWORD

Climate change characterized by escalating global temperatures and increasingly volatile precipitation regimes exerts a multifaceted pressure on agricultural ecosystems. These shifting climatic baselines alter the biogeographical distribution of plant pathogens and accelerate the evolutionary emergence of novel strains. Concurrently, the exacerbated environmental stress compromises host plant physiology, leading to a significant degradation of plant defense mechanisms and increased susceptibility to diseases. In response to the escalating threat of climate-fueled crop diseases, the Joint FAO/IAEA Centre has established a comprehensive Plant Health framework designed to safeguard global food systems through two core pillars:

- **Pathogen Containment:** Strengthening cross-border surveillance and early-detection networks to arrest pathogen dissemination, alongside implementing soil-management protocols to minimize regional pathogen loads.
- **Long-term Resilience:** Deploying innovative nuclear technologies to develop resistant crop varieties for both export markets and diverse local production systems, ensuring sustained food security and macroeconomic resilience.

High-throughput sequencing and bioinformatic analyses play an increasingly important role in supporting both pillars by enabling pathogen surveillance, characterization of improved germplasm, and genomic analysis of beneficial microorganisms.

This document represents a strategic effort to streamline and standardize the application of high-throughput sequencing in the dual context of crop improvement and biological disease control. Developed as a comprehensive technical framework, these guidelines are designed to facilitate the identification of genomic variation induced by gamma irradiation in both banana (*Musa* spp.) and their associated beneficial microorganisms.

Although the examples presented in this tutorial focus on banana improvement and beneficial microorganisms associated with disease management, the bioinformatic workflows described are broadly applicable to the identification of genomic variants, including single nucleotide polymorphisms (SNPs), insertions/deletions (indels), structural variants, and copy number variants associated with a wide range of agronomic and biological traits.

As global banana production faces intensifying pressure from evolving pathogens and climate instability, the integration of mutation breeding and advanced genomics has become essential. This guide provides a roadmap for navigating the key stages of modern sequencing workflows, from sample preparation and DNA extraction through library preparation, sequencing, quality assessment, assembly, and downstream genomic analyses tailored to specific research objectives.

Ultimately, this document is intended to serve as a practical resource for researchers and breeders in the IAEA and FAO Member States, providing guidance to ensure that sequencing projects are well designed and that resulting genomic data can be effectively used for downstream analyses in subsequent products.

The workflows described in this tutorial provide the foundation for downstream mutation discovery analyses.

Table of Contents

- I. Overview
- II. System requirements and software setup
- III. Datasets used in this tutorial
- IV. Sequencing data types

Part I. Long-read workflows - PacBio and Oxford Nanopore (ONT) data

- Quality control of long-read data
- Filtering long-read datasets
- De novo genome assembly

Part II. Working with short read data - Pair End Illumina

- Quality control of short-read data
- Trimming and filtering
- Read mapping to a reference genome
- Mapping quality assessment
- Variant quality assessment

Part III. Evaluating de novo assembly and maps to reference

Part IV. Discovering gamma irradiation-induced-mutations associated with a specific phenotype in banana

Part V. Analysis of microbial datasets

- De novo assembly of short reads (Illumina - small, bacteria, fungus)
- Discovering mutations induced with gamma irradiation associated to a specific phenotype in microorganisms

V. References

This document was produced under the framework of the Coordinated Research Project (CRP D23033), “An Integrative Approach to Enhance Disease Resistance Against Fusarium Wilt (*Foc* TR4) in Banana – Phase II”, led by the Plant Breeding and Genetics Section of the Joint FAO/IAEA Centre. We extend our gratitude to IAEA staff, as well as the international network of researchers and collaborating institutes whose collective efforts and insights contributed to the development of this work.

Bioinformatics tutorial for the analysis of genome data from gamma irradiated bananas and beneficial micro-organisms

I. Overview

This tutorial presents a workflow for the genome-level identification and analysis of genomic variants and candidate mutations induced by gamma irradiation in bananas and beneficial microorganisms.

The workflows include sequencing data quality assessment, read filtering, genome assembly, assembly evaluation, read mapping, variant discovery, and interpretation of candidate genomic regions associated with mutant phenotypes.

The workflow uses open-source bioinformatic tools that, in most cases, require the use of a command-line interface in Unix or Linux operating systems. You do not need advanced Linux expertise to follow this tutorial; however, familiarity with basic command-line operations and file management is recommended. However, if you need a refresher on basic commands, a list of commonly used Unix/Linux commands can be found here: [Unix/Linux Commands](#).

The tutorial is intended for researchers, breeders, molecular biologists, and laboratory scientists interested in applying genomic approaches to mutation breeding and microbial characterization.

All analyses in this tutorial were performed in a Unix/Linux environment. Some plots will be generated using R (<https://www.r-project.org/>).

Several workflows require moderate to high computational resources depending on genome size and sequencing depth.

II. System requirements and software setup

The following system configuration is recommended for successful completion of the workflows:

1. **Operating system:** The workflows described in this tutorial were developed and tested in a Unix/Linux environment.
2. **Software management:** Conda should be installed prior to starting the tutorial. If Conda is not installed, Miniconda can be downloaded from the following link: [Miniconda](#).
3. **Conda environments:** Conda environments containing the required bioinformatic tools should be properly created prior to starting the analyses and activated as needed

throughout the workflow. Instructions for creating and managing Conda environments can be found here: [Conda guide](#).

For simplicity and compatibility, each tool or set of known compatible tools used in this tutorial has been installed in a dedicated Conda environment, which should be activated before running the corresponding commands.

4. **Computational resources:** Computational requirements vary depending on dataset size and workflow. Long-read assembly of plant genomes may require substantially greater memory, storage, and processing capacity than short-read analyses or microbial datasets.

III. Datasets used in this tutorial

The datasets used in this tutorial are divided into two categories: (1) illustrative datasets used to demonstrate expected outputs and result interpretation, and (2) publicly available datasets that can be used for hands-on exercises and workflow replication.

- 1) Illustrative datasets: for illustrative purposes (i.e. to demonstrate workflow outputs and result interpretation, but not for direct hands-on analysis), examples from several sequencing projects are presented throughout the tutorial. These include:
 - PacBio HiFi sequencing data from gamma-irradiated banana lines.
 - Illumina paired-end sequencing data from irradiated Grande Naine banana populations
 - Sequencing datasets from beneficial microorganisms subjected to mutagenesis.
- 2) Publicly available datasets: for hands-on exercises, publicly available datasets have been selected to allow users to reproduce the analyses presented in this tutorial.
 - A long-read Oxford Nanopore (ONT) dataset, which is a subsample of a plant genome dataset can be obtained from here: [ONT data subset](#). The following command can be used to download this dataset:

```
wget \
https://zenodo.org/record/7055935/files/Eucalyptus\_subsample\_ten\_pc\_nano\_SRR7153076.fastq.gz
```

- For short-read analyses, paired-end Illumina datasets corresponding to Musa accuminata variety Novaria, SRR11579628 (gamma irradiated) and SRR11579627 (non-irradiated, control) were used and can be downloaded from the NCBI Sequence Read Archive (SRA) repository.. To download and convert the SRA files to FASTQ pair-end data, sra-tools application installed in a conda environment will be used.

```
Conda activate sra_env

#for the mutant:

Prefetch SRR11579628
Fasterq-dump -split-files SRR11579628

#for the control:

Prefetch SRR11579627
Fasterq-dump -split-files SRR11579627
```

- The reference genome used throughout the tutorial corresponds to *Musa acuminata* (AAA genome group; GCA_032878665.1_ASM3287866v1_genomic.fna.gz) and was selected because it represents a high-quality publicly available assembly suitable for demonstrating read mapping, variant discovery, and genome-scale analyses.. It can be directly downloaded from NCBI in FASTA format using the following command:

```
wget \
https://ftp.ncbi.nlm.nih.gov/genomes/all/GCA/032/878/665/GCA_032878
665.1_ASM3287866v1/GCA_032878665.1_ASM3287866v1_genomic.fna.gz
```

The datasets included in this tutorial were selected to represent common analytical scenarios encountered in mutation breeding and microbial genomics, including long-read assembly, short-read resequencing, mutant-versus-control comparisons, and microbial genome analysis. Although the examples focus on banana and beneficial microorganisms, the workflows can be adapted to other crops and microbial systems.

IV. Sequencing data types

Sequencing data generated by high-throughput sequencing platforms are typically provided in FASTQ format, which contains both the nucleotide sequence and a corresponding quality score for each base. FASTQ files represent the primary input for most genomic analyses and serve as the starting point for quality control, filtering, genome assembly, read mapping, and variant discovery workflows. Each read is represented as a four-line record:

1. A read identifier line beginning with “@” symbol
2. The nucleotide sequence
3. A separator line starting with the “+” symbol

4. A line of ASCII-encoded quality score string representing the confidence associated with each base call.

The structure of a FASTQ record is illustrated in the Figure 1.

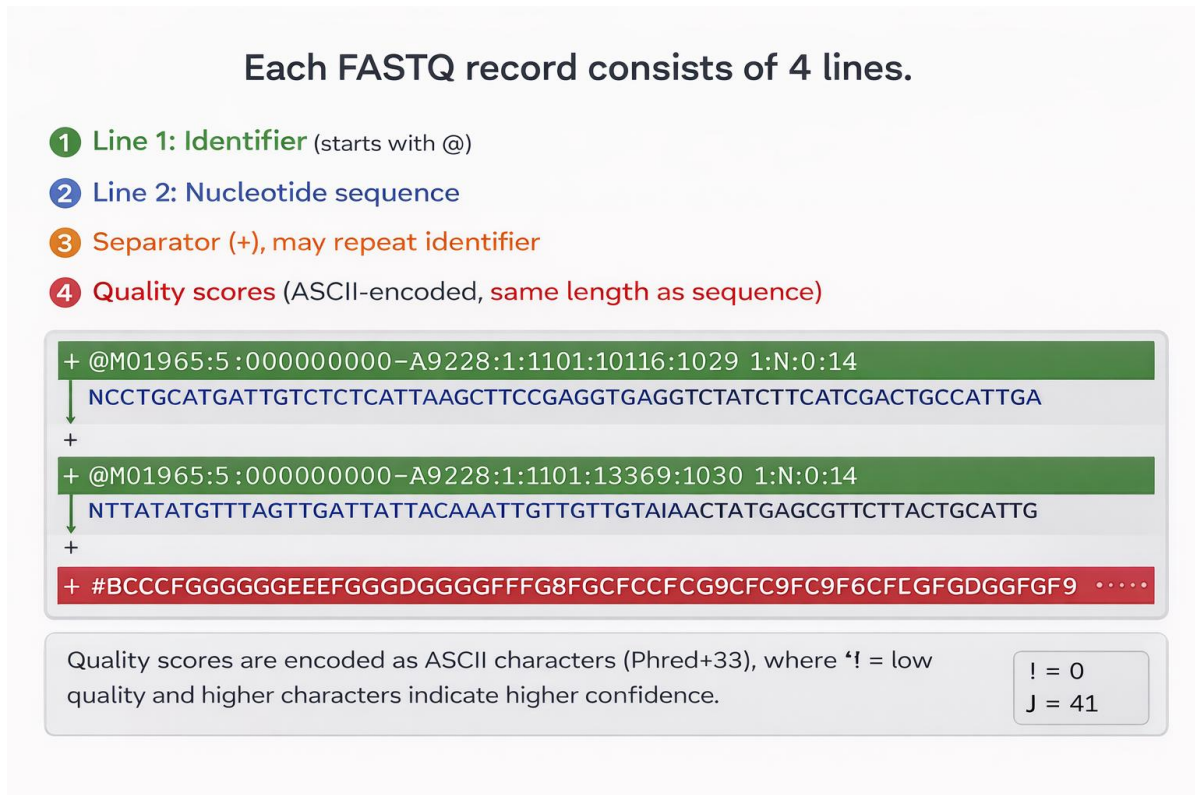


Figure 1. The structure of a *.fastq file - Each read is represented as a four-line record: A read identifier line beginning with “@” symbol, the nucleotide sequence, a separator line starting with the “+” symbol and a line of ASCII-encoded quality score string representing the confidence associated with each base call.

Quality scores are encoded using the Phred quality scoring system, where higher scores indicate greater confidence in the accuracy of the corresponding nucleotide call. These scores provide an estimate of the probability that a nucleotide has been incorrectly identified during sequencing.

Assessment of read quality is a critical first step in any genomic workflow. Low-quality reads can negatively affect downstream analyses, including genome assembly, read mapping, variant detection, and genome annotation. Consequently, quality control is routinely performed before proceeding with further analyses.

The following sections describe approaches for evaluating and improving sequencing data quality prior to downstream genomic analyses.

Part I. Long-read workflows - PacBio and Oxford Nanopore (ONT) data

The tools used in this section are NanoPlot, NanoFilt and Hifiasm. It is assumed that the required conda environment(s) containing these tools have already been created and activated prior to starting the analysis.

Quality control of long-read data

For quality assessment of long-read sequencing data, such as PacBio and ONT datasets, we will use [NanoPlot](#) (De Coster et al., 2018; De Coster and Rademakers, 2023).

For evaluating the quality of the FASTQ data, run the following commands depending on the technology used:

```
# For PacBio reads
NanoPlot --fastq pacbio_data_hifi.fastq.gz \
        -o pacbio_output_directory
        -t 60 --loglength

# For ONT reads
NanoPlot --fastq ont_reads.fastq.gz \
        -o ont_output_directory \
        -t 60 --loglength --N50
```

Where:

- fastq or fastq.gz (compressed version of FASTQ) are the input sequencing reads
- -o specifies the output directory
- -t specifies the number of threads
- --loglength plots read lengths on a logarithmic scale
- --N50 calculates and reports the N50 value, defined as the read length for which 50% of the total sequenced bases are contained in reads of that length or longer

Results

NanoPlot, described in the Guidelines for the identification of mutations using high-throughput sequencing: the case of banana and beneficial microbes ([docs/Guidelines for sequencing.pdf](#)). NanoPlot provides a rapid overview of sequencing yield, read length distribution, and quality metrics that are useful for determining whether a dataset is suitable for downstream analyses. NanoPlot generates several output files after running the command. The most relevant outputs include:

- 1) NanoStats (statistical summary of the sequencing data)
- 2) NanoPlot-report - interactive report containing quality metrics and graphical summaries.

- 3) Read-length distribution plots and histograms
- 4) Log file documenting program execution.

Results interpretation

The html report for the PacBio dataset: [PacBio NanoPlot report](#)

And the html report for the ONT dataset: [ONT NanoPlot report](#)

The graph - all the graphs are present in the html report. However, the most important graphs for the interpretation of results are shown below.

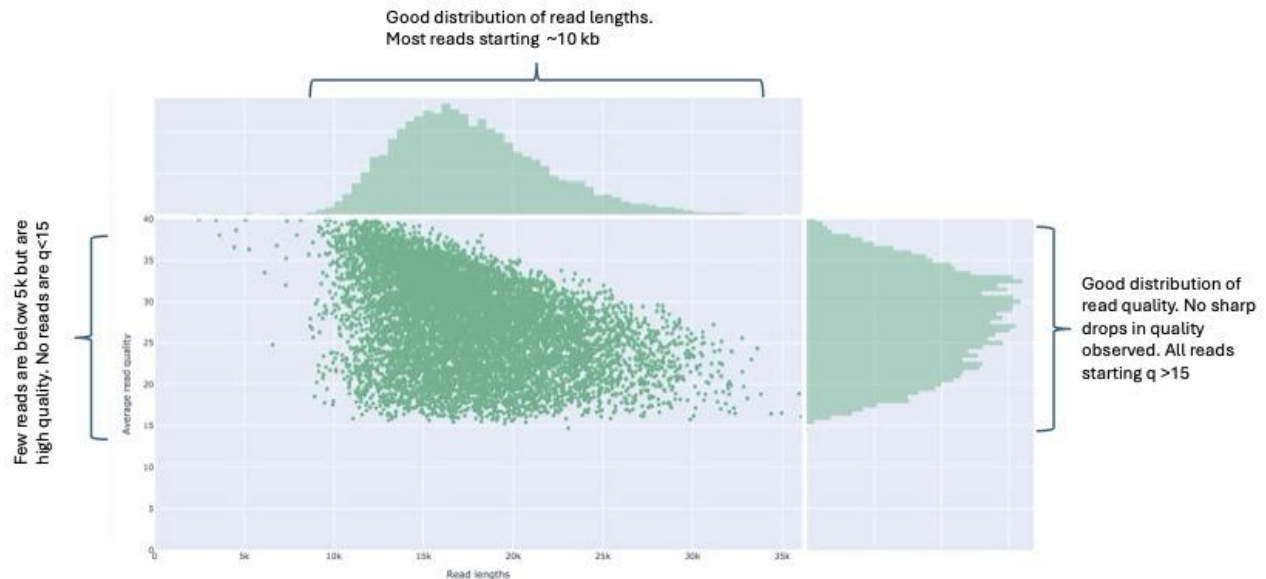


Figure 2. NanoPlot PacBio reads quality

The PacBio plot in Figure 2 shows high-quality sequencing dataset. Most reads cluster between ~10-25 kb, with relatively few short reads (< 5Kb). The read length distribution is smooth and continuous, without sharp peaks or drops suggesting consistent library preparation and sequencing performance. Most reads exhibit high quality scores, above 15, and no substantial decrease in quality is observed with increasing read length.

These characteristics are consistent with the expected performance of PacBio HiFi sequencing and indicate that the dataset is suitable for downstream analyses such as genome assembly and variant discovery.

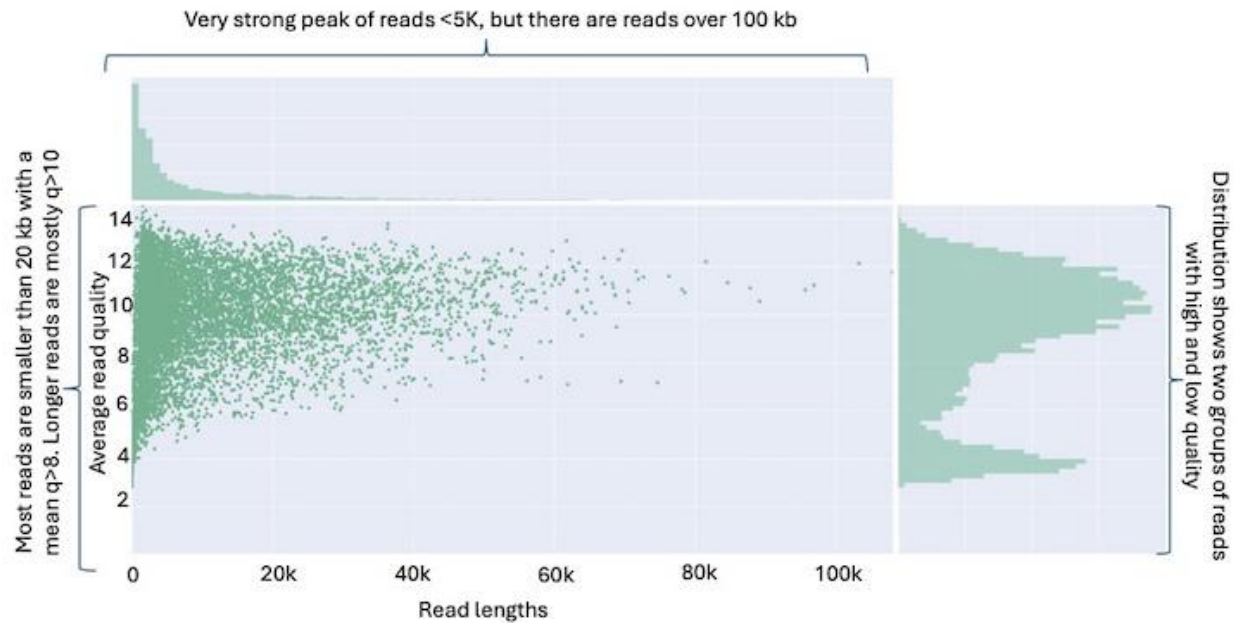


Figure 3. NanoPlot ONT read quality

The ONT plot in Figure 3 shows a typical long-read dataset characterized by a substantial proportion of shorter reads (<5 kb) together with a long-read tail extending beyond ~100 kb. Most reads cluster at moderate lengths, resulting in an intermediate N50 value.

The quality distribution is centered around Q10–12, with a secondary lower-quality population producing a bimodal pattern. A slight decline in quality with increasing read length is observed, which is expected for ONT datasets. Although the average read quality is lower than that typically observed for PacBio HiFi sequencing, the dataset remains suitable for downstream assembly and structural variant analyses provided sufficient sequencing depth is available.

The NanoPlot HTML report provides a comprehensive summary of sequencing statistics for the PacBio dataset. Selected metrics that are particularly informative for assessing read quality and sequencing performance are summarized in Table 1 and discussed below.

Table 1. PacBio data summary statistics

Metric	Observation
Number of reads	2,782,249
Median read length	17068.0
Mean read length	17597.6
N50	18039.0
Median quality	27.7
Mean quality	24.3

Reads >Q15	2,781,501
Reads >Q20	2,491,374
Reads >Q25	1,821,859
Reads >Q30	1,009,782

PacBio HiFi sequencing typically yield 4-8 million HiFi reads per SMRT cell (in the Revio/Sequel II). The number of reads obtained in this example is lower than some high-output runs; however, the adequacy of sequencing output should ultimately be evaluated in relation to genome size, ploidy level, and target sequencing depth. Median read length and mean read length are very similar, indicating a relatively consistent read-length distribution with limited influence from extreme values. Both metrics fall within the expected range for PacBio HiFi sequencing (approximately 15–20 kb) (Hon et al., 2020). In addition, the N50 value of approximately 18 kb indicates that 50% of the total sequenced bases are contained in reads of this length or longer, supporting the suitability of the dataset for downstream genome assembly.

The median quality score (27.7) and mean quality score (24.3) indicate that the majority of reads are of high quality, consistent with the quality distribution observed in the NanoPlot report. Although a small proportion of reads fall below Q20, most reads exceed this threshold. In the next step, quality filtering will be applied to remove lower-quality reads and improve the overall quality of the dataset used for assembly. The choice of filtering threshold should be guided by the sequencing platform and the intended downstream application.

The ONT dataset used in this tutorial represents a reduced subset of the original data and is provided primarily for demonstration purposes. All the statistics related to this dataset are presented in the html report.

Filtering long-read datasets

As observed in the QC section, a proportion of the PacBio HiFi reads have quality scores below Q20. To improve the overall quality of the dataset used for genome assembly, reads below the selected quality threshold can be removed using [NanoFilt](#) (De Coster et al., 2018; De Coster and Rademakers, 2023).

For filtering reads by quality, run the following command:

```
zcat pacbio_data_hifi.fastq.gz | NanoFilt -q 20 | gzip > \
pacbio_data_hifi_q20.fastq.gz
```

This command uses `zcat` to read the compressed FASTQ file (.fastq.gz)

and stream its contents directly to NanoFilt through the pipe operator (`|`). NanoFilt filters reads based on a minimum average quality score of Q20 and writes the retained reads to standard output. The filtered reads are then passed to `gzip`, which compresses the output and saves it as a new `fastq.gz` file (`pacbio_data_hifi_q20.fastq.gz`). This approach avoids creating large intermediate files and is commonly used when processing high-throughput sequencing data.

Once the filtered dataset has been generated, run the following command to determine how many reads were retained after filtering:

```
zcat pacbio_data_hifi_q20.fastq.gz | wc -l | awk '{print "Reads: "$1/4}'
```

This command counts the number of sequencing reads retained after filtering. The `zcat` command streams the contents of the compressed FASTQ file, `wc -l` counts the total number of lines, and `awk` divides this value by four because each FASTQ record consists of four lines (read identifier, sequence, separator, and quality scores).

For this dataset, the command reports 2,491,375 retained reads. Comparing this value with the original read count provides an estimate of the proportion of reads removed during quality filtering and helps assess the impact of the selected filtering threshold.

ONT datasets can also be filtered, one commonly used tool for this purpose is `Filtlong` (Wick, 2018), which allows filtering based on both read length and read quality.

For filtering ONT reads with `Filtlong` use the following command:

```
filtlong --min_length 1000 --min_mean_q 8 ONT_reads.fastq.gz > \
ONT_filtered.fastq
```

This command retains reads longer than 1,000 bp with a mean quality score greater than Q8. These thresholds are commonly used for ONT datasets; however, optimal filtering parameters will depend on sequencing quality, sequencing depth, and the intended downstream application.

De-novo genome assembly

Once the data set has been filtered and quality assessed, it can be used for de novo genome assembly. For Pacbio HiFi data, [Hifiasm](#) (Cheng et al., 2021) is the widely used assembler that supports haplotype-resolved assembly of each chromosome and performs particularly well for complex and highly heterozygous genomes such as banana.

Given the dataset consists of PacBio HiFi reads contained within a single FASTQ file, we will use the standard Hifiasm workflow for HiFi-only assembly. A basic command to start an assembly is shown below:

```
hifiasm -o pacbio_data_q20_hifiasm.asm -t 16 pacbio_data_hifi_q20.fastq.gz
```

The -o parameter specifies the output file prefix and -t indicates the number of processing threads. This is a basic command that allows to generate an initial k-mer distribution plot (Figure 4) that can be used to assess sequencing depth, heterozygosity, and the suitability of different assembly strategies.. The examples shown in Figure 4 represent two contrasting datasets: one derived from a highly heterozygous genome and the other from a largely homozygous genome.

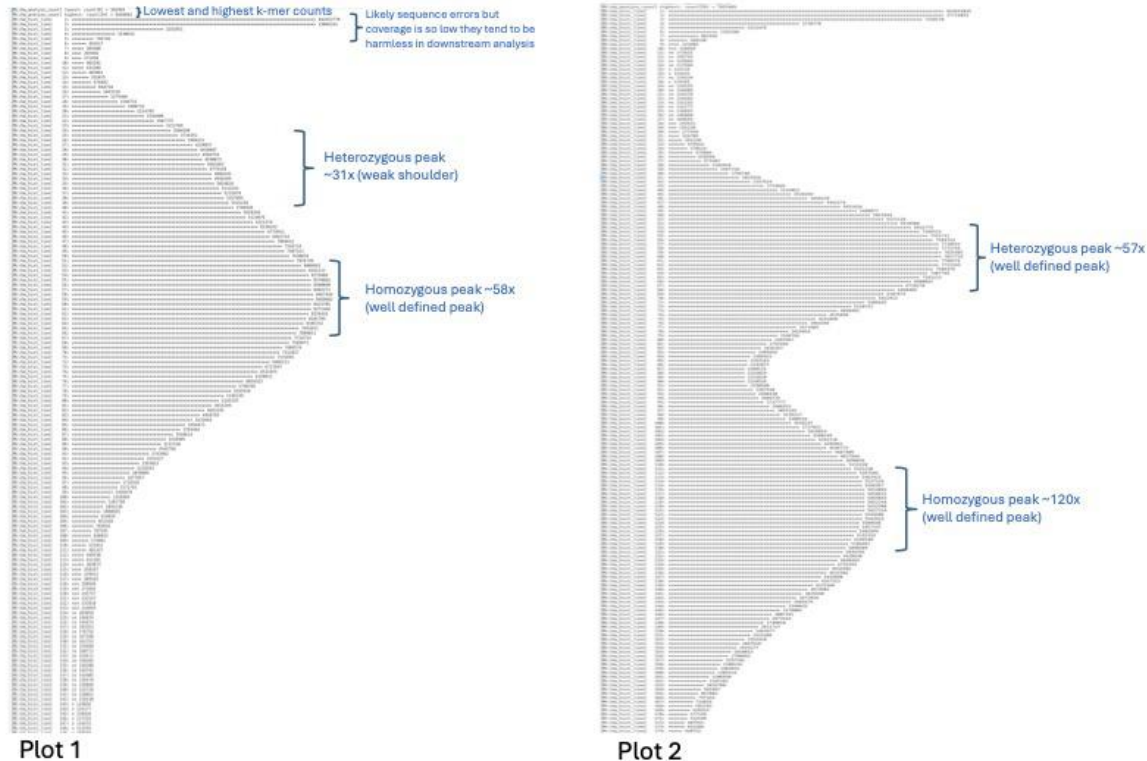


Figure 4. Hifiasm *de-novo* assembly plots

Once the k-mer distribution for the Pacbio read data has been evaluated, assembly parameters can be selected according to the level of heterozygosity and sequencing depth observed in the dataset.. Plot 1- shows a dominant peak with a weak shoulder, indicating low heterozygosity and therefore limited potential for haplotype phasing the assemble. The sample shown in Plot1 is likely to be an inbred or highly homozygous. Under these conditions, the assembly is expected to be largely collapsed into a consensus representation rather than producing well-resolved

haplotypes.. Plot 2 shows two well defined peaks corresponding to heterozygous and homozygous k-mers, indicating moderate heterozygosity and strong potential for haplotype-resolved assembly.. Both plots exhibit a gradually decaying tail, consistent with the presence of repetitive and multicopy sequences commonly observed in plant genome. Table 2 shows the interpretative differences between plots.

Table 2. Interpretative differences between Plot1 and Plot2

Feature	Plot 1	Plot 2	Interpretation
Heterozygosity peak depth	~31x	~51x	Single-copy k-mers present in one haplotype
Homozygosity peak depth	~58x	~120x	Single-copy k-mers present in both haplotypes
Heterozygosity : homozygosity ratio	~1:1.9	~1:2.0	Both consistent with diploid genomes
Peak separation	Overlapping (shoulder)	Well separated	Plot 2 has cleaner bimodality
Effective diploid depth	~58x	~120x	Plot 2 has higher sequencing depth (~2-fold)
Expected phasing quality	low-moderate	excellent	Higher depth and better peak separation improves hifiasm phasing.

Assembling method will depend on heterozygosity/homozygosity levels and amount of data. Plot 1 data is from a homozygous genome and reduced dataset. Plot 2 data is from a heterozygous genome and larger dataset.

Assembling Plot 1 data: To assemble the data from plot1, we use the following command:

```
hifiasm -f0 -o plot1_hifi_data.asm -t32 -l0 plot1_hifi_data.fq.gz
```

where -f0 is recommended for smaller genomes (< 3.0 Gb) and can improve computational efficiency; -l0 disables purge duplication which is recommended for homozygous genomes and -t32 specifies the number of processing threads. This assemble command will generate two main files:

- plot1_hifi_data.asm.p_ctg.gfa
- plot1_hifi_data.asm.a_ctg.gfa

corresponding to the primary (p_ctg) and alternative (a_ctg) assemblies (not included in primary assembly).

Assembling Plot 2 data: To assemble the data from plot2 we can use one of the following commands:

```
hifiasm -f0 -o plot2_hifi_data.asm -t 32 plot2_hifi_data.fq.gz

#or alternatively:

hifiasm -f0 -o plot2_hifi_data.asm -t 32 --dual-scaf plot2_hifi_data.fq.gz
```

For the second command, the --dual-scaf Hifiasm can improve assembly continuity by connecting contigs into longer scaffolds based on the graph structure. Not all datasets are suitable for dual scaffolding; however plot2 represents a dataset with high sequencing depth and clearer separation between heterozygous and homozygous k-mer peaks, making it well suited for haplotype-resolved assembly and the use of dual scaffolding.

In either assemble case, plot2-datasets, Hifiasm will generate three main files:

- plot2_hifi_data.asm.p_ctg.gfa
- plot2_hifi_data.asm.hap1_ctg.gfa
- plot2_hifi_data.asm.hap2_ctg.gfa

These files correspond to the primary assembly (p_ctg) and the two phased haplotype assemblies (hap1_ctg and hap2_ctg).. The primary assembly (p_ctg) can be viewed as a consensus representation derived from the two haplotypes, where continuity is prioritized by selecting a single path through the assembly graph. As a result, the primary assembly generally contains the longest and most contiguous sequences, while the phased haplotype assemblies (hap1_ctg and hap2_ctg) retain haplotype-specific information.

In addition to the output files listed above, Hifiasm generates several other auxiliary output files that contain information on assembly graphs, phasing, and assembly statistics. For a detailed description of all Hifiasm output files, refer to the Hifiasm documentation: [Hifiasm read the docs](#).

Once Hifiasm has completed the assembly, the "*_ctg.gfa" files need to be converted to FASTA format before downstream analysis can be performed. This can be achieved using the following command:

```
awk '/^S/{print ">"$2;print $3}' plot1_hifi_data.asm.p_ctg.gfa > \
plot1_hifi_data.p_ctg.fa
```

This command extracts the sequence entries (S) from the GFA file and converts them into standard FASTA format, which is required by most downstream genome analysis tools.

For additional Hifiasm information please refer to the Hifiasm documentation: [Hifiasm read the docs](#).

Part II - Working with short read data - Pair End Illumina

Quality control for short read data

This section describes how to assess the quality of short-read sequencing data generated using Illumina platforms. High-quality sequencing data are essential for reliable mutation discovery because sequencing errors, adapter contamination, and low-quality bases can lead to inaccurate read mapping and false-positive variant calls.

The examples presented in this section use two paired-end Illumina datasets. The first dataset corresponds to the triploid *Musa acuminata* var. Cavendish Grande-Naine (S30GY_01_R1.fq.gz and S30GY_01_R2.fq.gz), while the second dataset corresponds to *Musa acuminata* var. Cavendish (SRR11579628_R1.fq.gz and SRR11579628_R2.fq.gz). The tools used in this section are FastQC and Fastp. It is assumed that the required conda environment(s) containing these tools have already been created and activated prior to starting the analysis..

For quality assessment of short-read sequencing data such as Illumina FASTQ datasets, we will use FastQC (Andrews, 2010). Additional information about this tool can be found at [FastQC](#). The two example datasets differ in read length and sequencing depth, however the same quality control workflow can be applied to any FASTQ datasets. To evaluate the quality of the sequencing data, run the following commands:

```
fastqc SRR11579628_R1.fq.gz SRR11579628_R2.fq.gz

fastqc S30GY_01_R1.fq.gz S30GY_01_R2.fq.gz
```

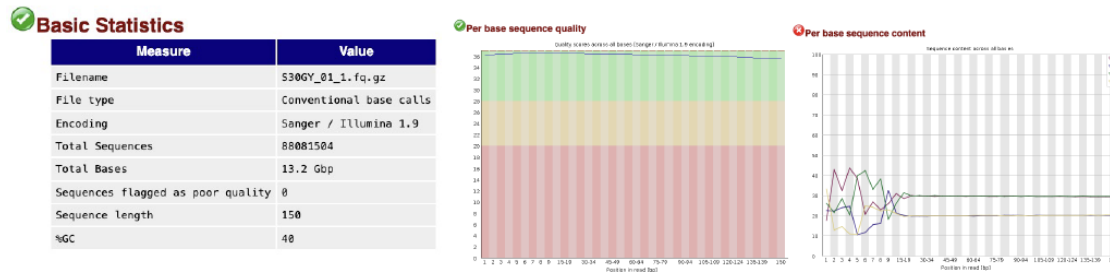
These commands run FastQC on both forward (R1) and reverse (R2) reads of each paired-end dataset. FastQC evaluates multiple quality metrics, including per-base sequence quality, sequence length distribution, GC content, adapter contamination, and sequence duplication levels..

FastQC generates separate reports for each read file (R1 and R2). The reports for example datasets can be found here: [S30GY_01_1_fastqc](#), [S30GY_01_2_fastqc](#), [SRR11579628_1_fastqc](#), [SRR11579628_2_fastqc](#)

A summary of the report is shown in Figure 5 and Figure 6. For datasets S30GY_01_R1 and S30GY_01_R2, the FastQC reports indicate overall high quality sequencing data with read lengths of 150 bp. The left panel summarizes key dataset statistics, while the middle panel shows per-base sequence quality. Most bases have quality scores above Q30 (green top panel)

across read length, indicating a low sequencing error rate. The per-base sequence content module (right panel) is flagged by FastQC because nucleotide frequencies are not perfectly balanced across all positions. However, such patterns are commonly observed in Illumina datasets and do not necessarily indicate a problem with the sequencing data.

S30GY_01_R1.fq.gz



S30GY_01_R2.fq.gz

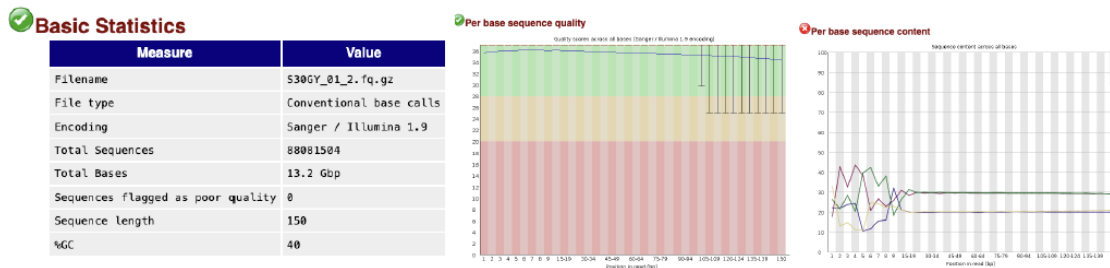


Figure 5. Representative metrics of a fastqc report for S30GY_01

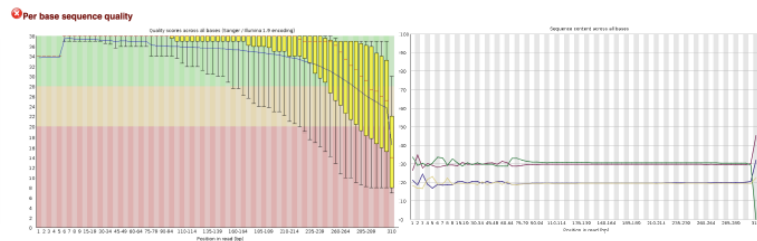
For datasets SRR11579628_R1 and SRR11579628_R2, the FastQC reports indicate overall good quality for short-reads of approximately 300 bp length. The left panel shows basic dataset statistics, and shows that the sequencing depth is lower than that observed for the S30GY_01 dataset. The middle panel shows that base quality of the reads remains high ($>Q30$) for the first ~150-200 bp but gradually declines toward the end of the reads, with some positions falling below $Q20$.

The per-base sequence content and adapter-content modules indicate the presence of residual adapter sequences and sequence composition bias toward the ends of the reads. Such patterns are commonly observed in longer Illumina reads and should be addressed prior to downstream analyses. Consequently, trimming and filtering are recommended before read mapping and variant discovery.

SRR11579628_1.fastq

Basic Statistics

Measure	Value
Filename	SRR11579628_1.fastq
File type	Conventional base calls
Encoding	Sanger / Illumina 1.9
Total Sequences	3131479
Total Bases	941.3 Mbp
Sequences flagged as poor quality	0
Sequence length	32–310
%GC	39



SRR11579628_2.fastq

Basic Statistics

Measure	Value
Filename	SRR11579628_2.fastq
File type	Conventional base calls
Encoding	Sanger / Illumina 1.9
Total Sequences	3131479
Total Bases	943.6 Mbp
Sequences flagged as poor quality	0
Sequence length	32–310
%GC	40

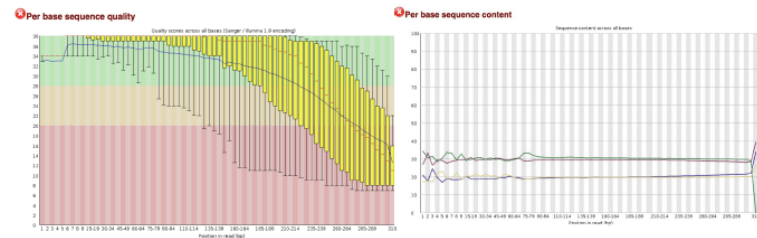


Figure 6. Representative metrics of a FastQC report for SRR11579628

Trimming and filtering data as needed

Based on the quality assessment performed in the previous step, adapter trimming and quality filtering are recommended for the SRR11579628 dataset. These steps help remove low-quality bases and residual adapter sequences that may interfere with downstream analyses such as read mapping and variant detection. Trimming and filtering will be performed using Fastp tool (Chen et al., 2018), an integrated tool for quality filtering, adapter trimming, and generation of quality-control reports.. Additional information for Fastp can be found on this page: [Fastp](#). The following command can be used to filter and trim both datasets:

```
fastp -i SRR11579628_1.fastq -I SRR11579628_2.fastq -o \
SRR11579628_filtered_1.fastq \
-O SRR11579628_filtered_2.fastq -q 20 --detect_adapter_for_pe \
-g -x -t 1 -h SRR11579628_fastp_report.html
```

In this command, both paired-end read files are processed simultaneously.. The `-i` and `-I` parameters specify the input files, while `-o` and `-O` define the filtered output files.. The quality parameter `(-q 20)` retains bases with quality scores of at least Q20 and `--detect_adapter_for_pe` enables automatic detection and removal of adapter sequences from paired-end reads.

The `-g` option removes poly-G tails, which are commonly observed in datasets generated on certain Illumina platforms such as NextSeq and NovaSeq. The parameter `-x` helps removing any low-complexity regions, including homopolymer tails that may arise from sequencing artifacts.

The `-t 1` parameter trims one base from the 5' end of each read. however this option is dataset-specific and may not be necessary in all cases..

Finally, `-h` and `-j` parameters generate html and json report, respectively. The HTML report is intended for visual inspection, while the JSON report provides a machine-readable summary that can be incorporated into automated analysis pipelines..

An example Fastp html report for the SRR11579628 dataset can be found here: [Fastp_report](#) If desired, FastQC can be re-run on the filtered/trimmed files to confirm that adapter contamination has been removed and that overall read quality has improved.

Mapping reads to a reference genome (short reads, large genomes, polyploid genomes)

The genome for *Musa acuminata* AAA Group (dessert banana) will be used as the reference genome for both datasets (Illumina 150 bp and Illumina 300 bp). The reference genome is typically distributed in compressed FASTA format (.fna.gz) and can be decompressed using the following command:

```
gunzip GCA_032878665.1_ASM3287866v1_genomic.fna.gz
```

Genome indexing

This section uses Minimap2 for read alignment and Samtools for manipulation and indexing of alignment files. Minimap2 was chosen for pipeline consistency and flexibility as it is an aligner that performs well across both short and long reads via presets. BWA-MEM2 or any other short read aligner would also have been appropriate.

It is assumed that the appropriate Conda environment containing these tools has already been activated.. In many Conda installations, Samtools is installed alongside Minimap2; however, users should verify that both programs are available in their environment before proceeding.

Before read alignment, the reference genome must be indexed using Minimap2. During indexing, Minimap2 generates a searchable representation of the genome based on k-mers/minimizers (short stretches of sequence) and their genomic positions, enabling rapid and memory-efficient alignment of sequencing reads. Minimap2 index the genome using the following command:

```
minimap2 -d GCA_032878665.1_ASM3287866v1_genomic.mmi \
GCA_032878665.1_ASM3287866v1_genomic.fna
```

The minimap2 will run the Minimap2 alignment tool. The -d option instructs Minimap2 to generate an index and specifies the name of the output index file, in this case the output file will be named:

GCA_032878665.1_ASM3287866v1_genomic.mmi. The final argument, GCA_032878665.1_ASM3287866v1_genomic.fna, is the input reference genome in FASTA format from which the index will be built.

Once the reference genome has been indexed, sequencing reads can be aligned to the reference genomes using the commands listed below. The resulting alignments are sorted and stored in BAM format, after which BAM index files are generated using Samtools to support efficient downstream analyses.

The SRR11579627 and SRR11579628 datasets are publicly available and can therefore be used for a hands-on exercises. Additional datasets included in this tutorial are provided for illustrative and comparative purposes.

```
minimap2 -ax sr --secondary=no -t 32 \
GCA_032878665.1_ASM3287866V1_genomic.mmi S30GY_01_filtered_1.fastq \
S30GY_01_filtered_2.fastq| samtools sort -o S30GY_01.sorted.bam

samtools index S30GY_01.sorted.bam

minimap2 -ax sr --secondary=no -t 32 \
GCA_032878665.1_ASM3287866V1_genomic.mmi S40GY_16_filtered_1.fastq \
S40GY_16_filtered_2.fastq| samtools sort -o S40GY_16.sorted.bam

samtools index S40GY_16.sorted.bam

minimap2 -ax sr --secondary=no -t 32 \
GCA_032878665.1_ASM3287866V1_genomic.mmi \ SA_GN_filtered_1.fastq \
SA_GN_filtered_2.fastq| samtools sort -o SA_GN.sorted.bam

samtools index SA_GN.sorted.bam

minimap2 -ax sr --secondary=no -t 32 \
GCA_032878665.1_ASM3287866V1_genomic.mmi \ SRR11579627_filtered_1.fastq
SRR11579627_filtered_2.fastq | samtools sort -o \ SRR11579627.sorted.bam

samtools index SRR11579627.sorted.bam

minimap2 -ax sr --secondary=no -t 32 \
GCA_032878665.1_ASM3287866V1_genomic.mmi \ SRR11579628_filtered_1.fastq
SRR11579628_filtered_2.fastq| samtools sort -o \ SRR11579628.sorted.bam

samtools index SRR11579628.sorted.bam
```

The command begins with minimap2, which performs alignment of sequencing reads to the indexed reference genome. The `-ax sr` option applies alignment parameters optimized for short-read sequencing data and outputs results in SAM format . The `--secondary=no` option disables reporting of secondary alignments, retaining only the primary alignments for each read. This option can simplify downstream analyses by reducing ambiguity in repetitive genomic regions.

The -t 32 parameter specifies the number of threads to 32 for parallel processing. The indexed reference genome file GCA_032878665.1_ASM3287866V1_genomic.mmi is followed by the paired-end FASTQ files representing the forward and reverse reads (S30GY_01_filtered_R1.fastq and S30GY_01_filtered_R2.fastq). The pipe operator (|) directs the alignment output directly to `samtools sort`, which sorts alignments by genomic coordinate and writes them to a BAM file using the -o option (S30GY_01.sorted.bam). Sorting is required before indexing and many downstream analyses.

The `samtools index` command generates a BAM index file using the sorted BAM file (S30GY_01.sorted.bam), enabling efficient retrieval of alignments from specific genomic regions without scanning the entire BAM file..

Part III - Evaluating de novo assembly and maps to reference

When a de novo genome assembly has been generated, its quality should be assessed before proceeding with downstream analyses. In this section, the *Musa acuminata* AAA Group reference genome is used as an example to demonstrate commonly used assembly quality metrics and evaluation approaches..

The quality of genome assembly is performed using [Quast](#) followed by [BUSCO](#) (Benchmarking Universal Single-Copy Orthologs) using the commands below.

```
quast.py GCA_032878665.1_ASM3287866v1_genomic.fna
```

The command begins with `quast.py`, which runs the QAST (Quality Assessment Tool for Genome Assemblies) which generates summary statistics describing assembly contiguity, fragmentation, completeness, and sequence composition. The input file is the genome assembly in FASTA format. Since no additional parameters are provided, QAST is executed using its default settings and generates a range of assembly statistics and reports describing genome size, contiguity, fragmentation, and sequence composition.

Selected metrics from the QAST output are summarized in Table 3 and are discussed below.

Table 3. The Quast output for the reference genome

Assembly	GCA_032878665.1_ASM3287866v1_genomic
# contigs (>= 0 bp)	2178
# contigs (>= 1000 bp)	2178
# contigs (>= 5000 bp)	167
# contigs (>= 10000 bp)	161
# contigs (>= 25000 bp)	141

# contigs (≥ 50000 bp)	98
Total length (≥ 0 bp)	1331403863
Total length (≥ 1000 bp)	1331403863
Total length (≥ 5000 bp)	1329349863
Total length (≥ 10000 bp)	1329304863
Total length (≥ 25000 bp)	1328934022
Total length (≥ 50000 bp)	1327412398
Largest contig	78871104
Total length	1331403863
GC (%)	39.26
N50	44298629
N90	30350507
auN	45790437.7
L50	12
L90	27
# N's per 100 kbp	31.90

QUAST Results Interpretation - The assembly exhibits high contiguity, with an N50 of 44.3 Mb and an L50 of 12, indicating that only 12 contigs account for 50% of the assembled genome sequence. The total assembly size is approximately 1.33 Gb which is slightly larger than a haploid Musa genome; which may indicate some redundancy. This may be confirmed in the next QC analysis. Although the assembly contains 2,178 contigs, the majority of the assembled sequence is contained within a relatively small number of large contigs (>50 kb), indicating a high degree of assembly contiguity and limited large-scale fragmentation. The low proportion of ambiguous bases (N's) suggests that relatively few unresolved regions remain within the assembly. The auN parameter is an indicator of contiguity, auN is the area under the Nx curve instead of just reporting the single N50 point, it integrates contiguity across the whole curve ($x = 0$ to 100); the higher the auN the more contiguous is the assembly. Overall, the QUAST metrics suggest that the assembly is highly contiguous and suitable for downstream analyses, although interpretation should be complemented by completeness assessments such as BUSCO.

A complementary approach for evaluating assembly quality is the use of BUSCO, which assesses genome completeness based on the presence of highly conserved genes expected within a given taxonomic lineage. The command used was the following:

```
busco -i ./GCA_032878665.1_ASM3287866v1_genomic.fna -l \
./busco_downloads/embryophyta_odb12/ \
-o ./GCA_busco_output -m genome --offline --force
```

The **Busco** command runs the BUSCO, a tool that assesses genome completeness by quantifying the presence of conserved single-copy orthologous genes expected to occur in a given lineage. These genes provide a standardized benchmark for comparing the completeness of genome assemblies. The **-i** option specifies the input genome assembly in FASTA format. The **-l** parameter defines the lineage dataset used for completeness assessment. In this example, the *embryophyta_odb12* dataset is used because the genome being analyzed belongs to the plant kingdom.. The backslash (\) is used as a line continuation character, allowing long commands to be split across multiple lines for improved readability. The **-o** option specifies the name of the output directory where results will be stored. The **-m genome** option specifies that the input consists of genomic DNA sequences. Alternative modes are available for transcriptome and protein datasets.. The **--offline** option instructs BUSCO to use locally available databases without attempting to download data during execution, whereas **--force** allows existing output directories with the same name to be overwritten.

BUSCO evaluates assembly completeness by comparing the assembly against a set of conserved plant orthologs expected to occur in most plant genomes. The results provide a standardized measure of gene-space completeness that can be compared across assemblies. The BUSCO results are shown in Table 4.

Table 4. Summary of BUSCO completeness metrics for the reference genome

Metric	Results
Complete BUSCOs	1990 (98.2 %)
Complete and single-copy BUSCOs	61 (3.0 %)
Complete and duplicated BUSCOs	1929 (95.2 %)
Errors (complete with stop codons)	18 (0.9 %)
Fragmented BUSCOs	22 (1.1 %)
Missing BUSCOs	14 (0.7 %)
Total BUSCO groups searched	2026 (100 %)
Number of scaffolds	2178
Number of contigs	6423
Total length	1331403863
Percent gaps	0.032%
Scaffolds N50	44 MB
Contigs N50	1 MB

Note: Note on scaffold/contig counts: BUSCO and QUAST define "contigs" differently, so their reported counts are not directly comparable. BUSCO internally splits input sequences at runs of N (gaps) to distinguish scaffolds from contigs, reporting both statistics separately. QUAST, by default, treats each FASTA record in the input file as a single unit (labeled "contigs" regardless of whether gaps are present) and does not split on N-runs unless the **--split-scaffolds** option is used. As a result, apparent discrepancies in contig/scaffold counts between the two tools reflect differing definitions rather than assembly errors.

BUSCO Results Interpretation- The BUSCO analysis using the *embryophyta_odb12* lineage dataset indicates that the banana assembly is highly complete, with 98.2% of expected conserved genes identified as complete. Only a small proportion of BUSCOs are fragmented (1.1%) or missing (0.7%), suggesting that most expected gene content is well represented within the assembly.

A large proportion of complete BUSCOs are classified as duplicated (95.2%), whereas only 3.0% are identified as single-copy orthologs. Interpretation of duplicated BUSCOs should be made in the context of genome biology, ploidy level, and assembly strategy. Elevated duplication levels may reflect biological features of the genome, such as segmental duplications or retained haplotypes, as well as assembly-related factors including incomplete haplotype collapsing. A small proportion of BUSCO genes (0.9%) contain internal stop codons, which may result from minor assembly or polishing errors, annotation limitations or genuine pseudogenization events. However, the observed frequency is low and does not substantially affect the overall assessment of assembly quality.. Overall, the assembly indicate a highly complete assembly with strong representation of expected gene content. Depending on the intended downstream application, additional investigation of the observed duplication levels may be warranted to determine whether further assembly refinement is required.

Evaluating genome mapping of Illumina reads to reference genome

Following read alignment, it is important to assess the quality of the mapping before proceeding to variant discovery using samtools and bcftools ([samtools/bcftools](#)). Alignment-based quality control metrics generated using samtools provide information on mapping efficiency, read pairing, and genome coverage. Variant-based metrics generated using bcftools provide an additional assessment of whether the alignments produce biologically consistent variant calls. Together, these metrics help identify potential technical artifacts and determine whether the data are suitable for downstream analyses such as SNP, indel, and mutation discovery.

Alignment QC with Samtools

Alignment-based quality control metrics were generated using samtools. Key metrics include the total number of reads, percentage of mapped reads, proportion of properly paired reads, mean sequencing depth, and breadth of genome coverage. Together, these statistics provide an assessment of alignment quality and the extent to which the reference genome is represented by the sequencing data.

```
samtools flagstat S30GY_01.sorted.bam > S30GY_01.sorted.flagstat.txt
samtools stats S30GY_01.sorted.bam > S30GY_01.sorted.stats
samtools depth -a S30GY_01.sorted.bam > S30GY_01.sorted.depth.txt
samtools stats S30GY_01.sorted.bam | grep ^MQ > S30GY_01.sorted.mq.txt
```

```
samtools depth S30GY_01.sorted.bam | awk '{if($3>0) c++} END {print c/NR}'>
S30GY_01.sorted.breadth.txt

samtools flagstat S40GY_16.sorted.bam > S40GY_16.sorted.flagstat.txt
samtools stats S40GY_16.sorted.bam > S40GY_16.sorted.stats
samtools depth -a S40GY_16.sorted.bam > S40GY_16.sorted.depth.txt
samtools stats S40GY_16.sorted.bam | grep ^MQ > S40GY_16.sorted.mq.txt
samtools depth S40GY_16.sorted.bam | awk '{if($3>0) c++} END {print c/NR}'>
S40GY_16.sorted.breadth.txt

samtools flagstat SA_GN.sorted.bam > SA_GN.sorted.flagstat.txt
samtools stats SA_GN.sorted.bam > SA_GN.sorted.stats
samtools depth -a SA_GN.sorted.bam > SA_GN.sorted.depth.txt
samtools stats SA_GN.sorted.bam | grep ^MQ > SA_GN.sorted.mq.txt
samtools depth SA_GN.sorted.bam | awk '{if($3>0) c++} END {print c/NR}'>
SA_GN.sorted.breadth.txt

samtools flagstat SRR11579627.sorted.bam > SRR11579627.sorted.flagstat.txt
samtools stats SRR11579627.sorted.bam > SRR11579627.sorted.stats
samtools depth -a SRR11579627.sorted.bam > SRR11579627.sorted.depth.txt
samtools stats SRR11579627.sorted.bam | grep ^MQ > SRR11579627.sorted.mq.txt
samtools depth SRR11579627.sorted.bam | awk '{if($3>0) c++} END {print
c/NR}' > SRR11579627.sorted.breadth.txt

samtools flagstat SRR11579628.sorted.bam > SRR11579628.sorted.flagstat.txt
samtools stats SRR11579628.sorted.bam > SRR11579628.sorted.stats
samtools depth -a SRR11579628.sorted.bam > SRR11579628.sorted.depth.txt
samtools stats SRR11579628.sorted.bam | grep ^MQ > SRR11579628.sorted.mq.txt
samtools depth SRR11579628.sorted.bam | awk '{if($3>0) c++} END {print
c/NR}' > SRR11579628.sorted.breadth.txt
```

The commands above generate complementary alignment quality metrics. `samtools flagstat` summarizes mapping statistics, including mapped and properly paired reads. `samtools stats` provides detailed sequencing and alignment statistics such as insert-size distributions and mismatch rates. `samtools depth` calculates per-base coverage across the reference genome, allowing estimation of sequencing depth and breadth of coverage. The mapping-quality distribution is extracted from the `samtools stats` output using `grep ^MQ`, providing an indication of alignment confidence.

Alignment QC with bcftools

Variant-based quality control metrics were derived from VCF files generated using `bcftools`. In addition to assessing data quality, these variants form the basis for downstream mutation discovery. Key metrics include the total number of variants, the numbers of SNPs and indels, the transition/transversion (Ts/Tv) ratio, the frequency of multiallelic sites, and allele-frequency distributions. SNPs correspond to single base substitutions relative to the reference genome,

while indels represent small insertions or deletions. The Ts/Tv ratio measures the proportion of transition mutations ($A \leftrightarrow G$ and $C \leftrightarrow T$, substitutions between bases of the same chemical class) to transversion mutations (purine $\leftrightarrow$ pyrimidine substitutions; $A \leftrightarrow C$, $A \leftrightarrow T$, $G \leftrightarrow C$, and $G \leftrightarrow T$). Because transitions occur more frequently in most genomes, the Ts/Tv ratio serves as an indicator of variant call quality, with expected values typically around 2.0-2.3. Multiallelic sites denote genomic positions with more than one alternate allele, reflecting either biological variation or local sequence complexity, and multiallelic SNPs represent the subset involving only SNP alleles. Allele frequency (AF) metrics distinguish between low-frequency variants ($AF \approx 0$), which may reflect rare alleles or low-confidence calls, and high-frequency variants ($AF \approx 1$), which typically represent fixed differences from the reference genome.

```
bcftools mpileup -Ou -f GCA_032878665.1_ASM3287866v1_genomic.fna
S30GY_01.sorted.bam | bcftools call -mv -Ov -o S30GY_01.sorted.vcf
bcftools stats S30GY_01.sorted.vcf > S30GY_01.sorted.vcf.stats.txt
bcftools query -f '%CHROM\t%POS\n' S30GY_01.sorted.vcf > \
S30GY_01.sorted.snp_positions.txt

bcftools mpileup -Ou -f GCA_032878665.1_ASM3287866v1_genomic.fna
S40GY_16.sorted.bam | bcftools call -mv -Ov -o S40GY_16.sorted.vcf
bcftools stats S40GY_16.sorted.vcf > S40GY_16.sorted.vcf.stats.txt
bcftools query -f '%CHROM\t%POS\n' S40GY_16.sorted.vcf > \
S40GY_16.sorted.snp_positions.txt

bcftools mpileup -Ou -f GCA_032878665.1_ASM3287866v1_genomic.fna
SA_GN.sorted.bam | bcftools call -mv -Ov -o SA_GN.sorted.vcf
bcftools stats SA_GN.sorted.vcf > SA_GN.sorted.vcf.stats.txt
bcftools query -f '%CHROM\t%POS\n' SA_GN.sorted.vcf > \
SA_GN.sorted.snp_positions.txt

bcftools mpileup -Ou -f GCA_032878665.1_ASM3287866v1_genomic.fna
SRR11579627.sorted.bam | bcftools call -mv -Ov -o SRR11579627.sorted.vcf
bcftools stats SRR11579627.sorted.vcf > SRR11579627.sorted.vcf.stats.txt
bcftools query -f '%CHROM\t%POS\n' SRR11579627.sorted.vcf > \
SRR11579627.sorted.snp_positions.txt

bcftools mpileup -Ou -f GCA_032878665.1_ASM3287866v1_genomic.fna
SRR11579628.sorted.bam | bcftools call -mv -Ov -o SRR11579628.sorted.vcf
bcftools stats SRR11579628.sorted.vcf > SRR11579628.sorted.vcf.stats.txt
bcftools query -f '%CHROM\t%POS\n' SRR11579628.sorted.vcf > \
SRR11579628.sorted.snp_positions.txt
```

The first command begins with `bcftools mpileup`, which summarizes the aligned reads at each genomic position and calculates genotype likelihoods relative to the reference genome. The `-Ou` option produces uncompressed BCF output that can be streamed directly to the next command without creating an intermediate file. The `-f` option specifies the reference genome in FASTA format against which variants will be identified. The pipe (`|`) passes the output to `bcftools call`,

where the `-m` option enables multiallelic variant calling and `-v` restricts the output to variant sites only. The `-Ov` option specifies VCF output format, and `-o` defines the name of the output VCF file containing the detected SNPs and indels.

The second command, `bcftools stats`, generates summary statistics for the VCF file, including variant counts, SNP and indel frequencies, transition/transversion ratios, and other metrics useful for variant-based quality assessment.

The third command, `bcftools query -f '%CHROM\t%POS\n'`, extracts the chromosome and genomic position of each detected variant. These coordinates can be used for downstream analyses such as mutation distribution studies, comparison of variants among samples, or visualization of variant locations across the genome.

Alignment-based quality control results

Table 5 summarizes alignment-based quality control metrics for the two datasets. Dataset 1 comprises Control GN, mutant 30GY and mutant 40GY, whereas Dataset 2 comprises SRR11579627 (control) and SRR11579628 (mutant).

Table 5. Summary of alignment-based QC

Dataset	1			2	
Metric	Control (GN)	30GY	40GY	SRR11579627	SRR11579628
Total reads	82.2 M	175.3 M	158.5 M	5.16 M	5.46 M
Mapped (%)	98.77%	99.07%	99.13%	99.48%	99.42%
Properly paired (%)	95.87%	96.81%	96.59%	93.13%	93.81%
Singletons (%)	0.33%	0.27%	0.28%	0.13%	0.12%
Different chr pairs (~%)	~1.6%	~1.3%	~1.4%	~3.6%	~3.6–3.7%
Mean depth	8.97	19.20	17.35	1.15	1.19
Breadth of coverage	0.9997	0.997	0.9997	0.9990	0.9990

Interpretation of alignment-based QC using samtools flagstat and depth calculations –

Alignment-based QC highlights differences in sequencing depth between two datasets while confirming overall high alignment quality across all samples. Dataset 1 contains substantially higher sequencing depth, with higher mean coverage ranging from $\sim 9\times$ to $\sim 19\times$, compared to Dataset 2 ($\sim 1\times$ coverage). Despite these differences, mapping rates are consistently high across all samples ($>98.7\%$), indicating efficient alignment of reads to the reference genome and suggesting that the reference genome is appropriate for these analyses.

Properly paired reads account for approximately 93-97% of mapped reads across all samples, indicating generally good library quality and alignment accuracy. The proportion of singleton reads is low (<0.4%) in all datasets, further supporting the reliability of the alignments. Dataset 2 shows a slightly higher proportion of read pairs mapping to different chromosomes (~3.6%) compared with Dataset 1 (~1.3-1.6%). This may reflect differences in genome structure, repetitive sequence content, library characteristics, or alignment ambiguity and should therefore be interpreted cautiously.

Breadth of coverage is high in all samples (>99.7%), indicating that most regions of the reference genome are represented in the sequencing data, even in the lower-depth dataset. Collectively, these results demonstrate that the sequencing reads align well to the reference genome and that the resulting alignments are suitable for downstream analyses, including variant discovery and comparative genomic analyses.

Variant-based quality control results

Table 6 summarizes the information from the QC analysis for the two datasets. Dataset 1: Control GN, mutant 30GY, mutant 40GY. Dataset 2: SRR11579627 (control) and SRR11579628 (mutant).

Table 6. Summary of variant-based QC

Dataset	1			2	
Metric	Control (GN)	30GY	40GY	SRR11579627	SRR11579628
Total variants	2459545	3060498	3053259	940269	1042891
SNPs	2309776	2860011	2850670	902774	999085
Indels	149769	200487	202589	37495	43806
Ts/Tv ratio	2.27	2.25	2.25	2.14	2.19
Multiallelic sites	1611	2752	2548	708	738
Multiallelic SNPs	307	434	470	51	73
AF $\approx$ 0 SNPs	1716333	2190846	2194173	219235	279185
AF $\approx$ 1 SNPs	593750	669599	656967	683590	719973

Interpretation of QC based on bcftools statistics - Comparison of variant statistics across the control and mutant samples shows a substantial increase in both SNPs and indels in the irradiated genomes. Despite this increase, all samples display consistent Ts/Tv ratios (~2.1) and similar

allele frequency distributions, indicating stable and reliable mapping and variant calling. These results suggest that the observed differences in variant counts are driven by biological effects of mutagenesis rather than with technical biases.

Variant-based QC also illustrates the influence of sequencing depth on variant discovery. Dataset 1 (control and mutant samples) exhibited substantially higher numbers of total variants, SNPs, and indels than Dataset 2, reflecting its greater sequencing depth and increased sensitivity for variant detection. In contrast, Dataset 2 showed lower variant counts and slightly reduced Ts/Tv ratios, consistent with the reduced ability of lower-coverage sequencing to detect rare variants. Nevertheless, Ts/Tv values remained within expected biological ranges across all samples, supporting the overall reliability of the variant calls.

The number of multiallelic sites remained low in all datasets, indicating minimal contribution from alignment artifacts. Allele frequency distributions further highlighted the effect of sequencing depth, with Dataset 2 containing fewer low-frequency variants and a relative enrichment of high-frequency variants. Within Dataset 1, mutant samples displayed higher variant counts relative to the control, consistent with the expected effects of induced mutagenesis. Overall, these results demonstrate that while sequencing depth influences the number and distribution of detected variants, key quality metrics remain stable, supporting robust downstream variant discovery and comparative analysis.

Part IV - Discovering gamma irradiation-induced-mutations associated with a specific phenotype in banana

Pre-processed datasets

We will continue working with the SRR11579627 (non-mutant control) and SRR11579628 (mutant) datasets. The raw sequencing files will no longer be used, as the required processed files were generated in the previous module. Specifically, we will use the BAM files obtained after aligning sequencing reads to the reference genome GCA_032878665.1_ASM3287866v1_genomic.

The objective of this section is to identify genomic changes that may have been induced by gamma irradiation. A stepwise workflow is presented to compare the mutant and control genomes and identify candidate regions associated with the observed phenotype.

Depth-based coverage scan

Large genomic changes such as deletions, duplications, and copy-number variations can alter local sequencing depth. Therefore, comparing genome-wide coverage patterns between the mutant and control provides an initial approach for identifying candidate regions affected by gamma irradiation.

The first step is to generate genomic windows across the reference genome. This requires indexing the reference genome using samtools and creating fixed-size genome bins using bedtools. These bins will subsequently be used to compare sequencing coverage between the control and mutant samples. We will use samtools and bedtools in this section. Samtools will be used to index the reference genome, whereas bedtools will be used to generate genomic windows and calculate sequencing coverage across those windows.

```
#Create reference genome bins

samtools faidx GCA_032878665.1_ASM3287866v1_genomic.fna && \
cut -f1,2 GCA_032878665.1_ASM3287866v1_genomic.fa.fai > \
GCA_032878665.1_ASM3287866v1_genomic.txt

bedtools makewindows -g GCA_032878665.1_ASM3287866v1_genomic.txt -w 50000 >\
GCA_032878665.1_ASM3287866v1_genomic.50kb.bed

sort -k1,1 -k2,2n GCA_032878665.1_ASM3287866v1_genomic.50kb.bed > \
GCA_032878665.1_ASM3287866v1_genomic.50kb.sorted.bed
```

The samtools faidx command indexes the reference genome FASTA file and generates a .fai index containing the names and lengths of all reference sequences. The cut -f1,2 command extracts this information into a two-column genome size file that can be used by bedtools. The bedtools makewindows command divides the genome into fixed 50 kb windows in BED format. Finally, sort -k1,1 -k2,2n sorts the windows by chromosome and genomic position, producing an ordered set of genomic intervals suitable for downstream coverage analyses.

Coverage per sample

Coverage must be calculated separately for each sample (control and mutant) using the same genomic window file. Both control and mutant bam files must be coordinate-sorted prior to running bedtools coverage command. The sorted versions of the files were generated during the read mapping to the reference genome workflow described in the previous section

The following commands calculates the average read depth across predefined 50 kb genomic windows for the control and mutant samples.

```
bedtools coverage \
  -a GCA_032878665.1_ASM3287866v1_genomic.50kb.sorted.bed \
  -b SRR11579627.sorted.bam \
  -mean \
  -sorted \
  -g GCA_032878665.1_ASM3287866v1_genomic.txt \
  > SRR11579627.depth.50k.bed

bedtools coverage \
  -a GCA_032878665.1_ASM3287866v1_genomic.50kb.sorted.bed \
  -b SRR11579628.sorted.bam \
  -mean \
  -sorted \
  -g GCA_032878665.1_ASM3287866v1_genomic.txt \
  > SRR11579628.depth.50k.bed
```

The **-a** option provides the BED file containing the predefined genome windows, whereas **-b** specifies the sorted BAM file containing the aligned sequencing reads. The **-mean** parameter instructs bedtools to calculate the average sequencing depth within each genomic window rather than reporting read counts. The **-sorted** option assumes that both the BED and BAM files are sorted, improving computational performance, while the **-g** parameter provides genome size generated in the previous step to ensure consistent chromosome ordering. The output is written to a file (SRR11579627.depth.50k.bed) containing one record per genomic window, including the chromosome coordinates and the corresponding mean depth of coverage. This file forms the

basis for comparing genome-wide coverage patterns between the control and mutant samples in subsequent analysis.

Regions with similar coverage in both samples are expected to have comparable mean depth values, whereas regions showing reduced or increased coverage in the mutant may indicate candidate structural changes, such as deletions or duplications, that warrant further investigation.

Compute the ratio of sequencing depth between control and mutant

This step compares sequencing depth between the mutant and control samples by calculating the $\log_2$ ratio of for each genomic window. The $\log_2$ transformation centers equal coverage at zero and represents proportional changes symmetrically, making deletions and duplications easier to identify visually. The R script takes as input the two depth files generated previously (one for the control and one for the mutant), reads them into R, and assigns column names corresponding to chromosome coordinates and mean sequencing depth values. The two datasets are then merged by matching the genomic windows (chr, start, end) to ensure direct comparison of the same regions. For each window, a $\log_2$ ratio is calculated as $\log_2((\text{depth_mut} + 1)/(\text{depth_ctrl} + 1))$, where a pseudo count of one is added to avoid division by zero in windows with no sequencing coverage. The resulting values indicate relative coverage differences, with values close to zero representing similar sequencing depth between the two samples, negative values indicating reduced coverage in the mutant, and positive values indicating increased coverage in the mutant. The final table is written to an output file and used for downstream visualization and identification of regions with potential structural variation. The resulting output file is then used by a second R script to generate genome-wide plots of coverage differences between the mutant and control samples.

```
#!/usr/bin/env Rscript

args <- commandArgs(trailingOnly = TRUE)

if (length(args) < 3) {
  stop("Usage: Rscript depth_ratio.R <control.depth.bed> <mutant.depth.bed> <output.txt>")
}

control_file <- args[1]
mutant_file  <- args[2]
output_file  <- args[3]

# Read files
control <- read.table(control_file, header = FALSE)
mutant  <- read.table(mutant_file, header = FALSE)

colnames(control) <- c("chr", "start", "end", "depth_ctrl")
colnames(mutant)  <- c("chr", "start", "end", "depth_mut")
```

```
# Merge by genomic window
df <- merge(control, mutant, by = c("chr", "start", "end"))

# Compute log2 ratio
df$ratio <- log2((df$depth_mut + 1) / (df$depth_ctrl + 1))

# Save
write.table(df, output_file, sep = "\t", row.names = FALSE, quote = FALSE)
```

The next step generates genome-wide plots of the $\log_2$ coverage ratio for each chromosome or haplotype. The R script reads the previously computed ratio file and, using the ggplot2 package, iterates over each chromosome to create individual plots. For each chromosome, the $\log_2(\text{mutant/control})$ ratio is plotted genomic position, allowing visualization of coverage differences across the genome. The resulting plots are saved as separate PDF files, one per chromosome, using a user-defined prefix. These visualizations are used to identify broad genomic regions with consistent shifts in coverage, which may indicate structural variation such as deletions, duplications or copy number variations.

```
#!/usr/bin/env Rscript

args <- commandArgs(trailingOnly = TRUE)

if (length(args) < 2) {
  stop("Usage: Rscript plot_depth.R <depth_ratio.txt> <output_prefix>")
}

input_file <- args[1]
prefix <- args[2]

library(ggplot2)

df <- read.table(input_file, header = TRUE)

for (c in unique(df$chr)) {
  sub <- df[df$chr == c, ]

  p <- ggplot(sub, aes(x = start, y = ratio)) +
    geom_line(linewidth = 0.3) +
    theme_bw() +
    ggtitle(c) +
    xlab("Position") +
    ylab("log2(mutant/control)")

  ggsave(paste0(prefix, "_", c, ".pdf"), plot = p, width = 10, height = 4)}
```

Evaluating the resulting plots.

A total of 33 plots were generated, representing the three haplotypes of each of the 11 banana chromosomes. At this stage, the objective is to identify broad and sustained shifts in sequencing depth between the mutant and control samples. Figure 7 shows an example of a chromosome that **DOES NOT** contain a candidate structural variant, which is observed in the majority of the haplotypes analyzed.

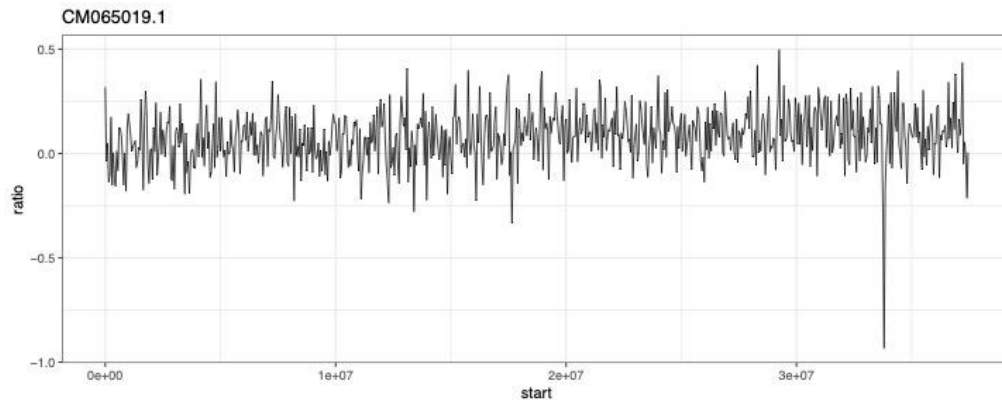


Figure 7. $\text{Log}_2(\text{mutant/control})$ sequencing depth ratio across haplotype CM065019.1

The region shown in Figure 7 is not considered a candidate for structural variation because the $\text{log}_2(\text{mutant/control})$ ratio remains largely centered around zero across the chromosome, indicating similar sequencing depth between the mutant and control samples. Although occasional sharp spikes are observed, these are isolated and do not extend across consecutive genomic windows, suggesting that they are more likely due to local noise, mapping artifacts, or repetitive sequences than true structural variation. In contrast, true candidate regions exhibit broad and continuous shifts in coverage that span multiple consecutive windows, often extending over several megabases. Because this chromosome lacks any sustained deviation from the baseline, it does not provide evidence for large-scale structural variation or copy number changes.

Among the 33 haplotypes analyzed, three candidate regions exhibiting sustained coverage shifts were identified on haplotypes CM065024.1, CM065025.1, and CM065034.1. The corresponding plots for three candidate regions are shown in Figure 8-10 and the results for all 33 haplotypes are summarized in Table 7.

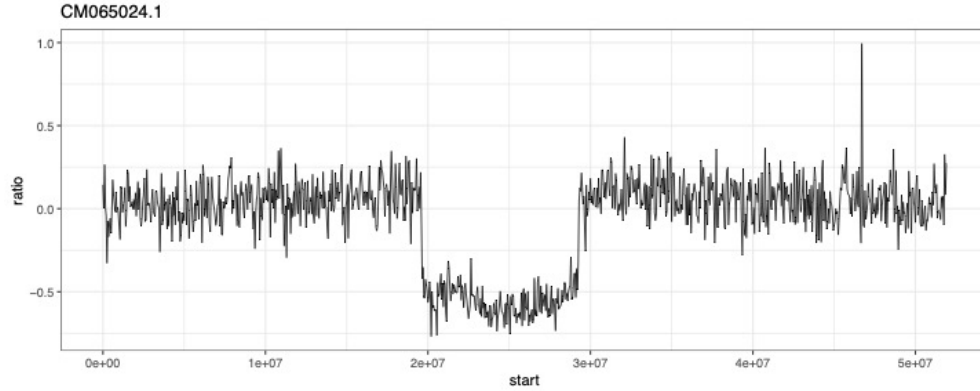


Figure 8. $\text{Log}_2(\text{mutant/control})$ sequencing depth ratio across haplotype CM65024.1

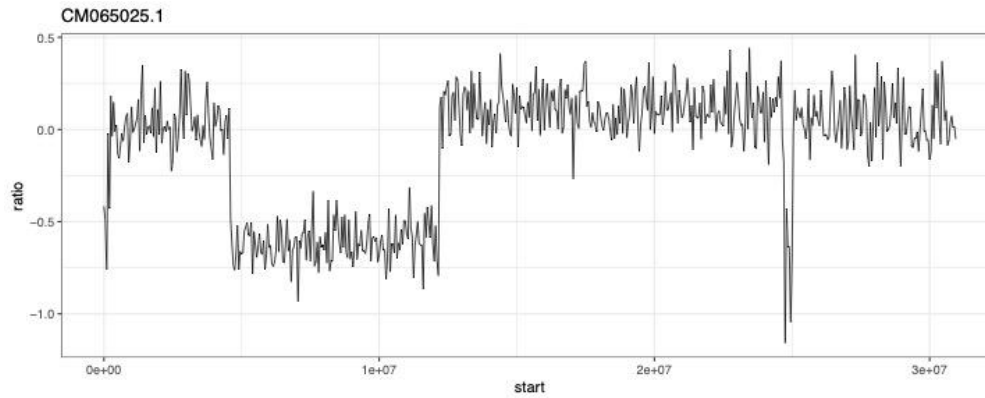


Figure 9. $\text{Log}_2(\text{mutant/control})$ sequencing depth ratio across haplotype CM65025.1

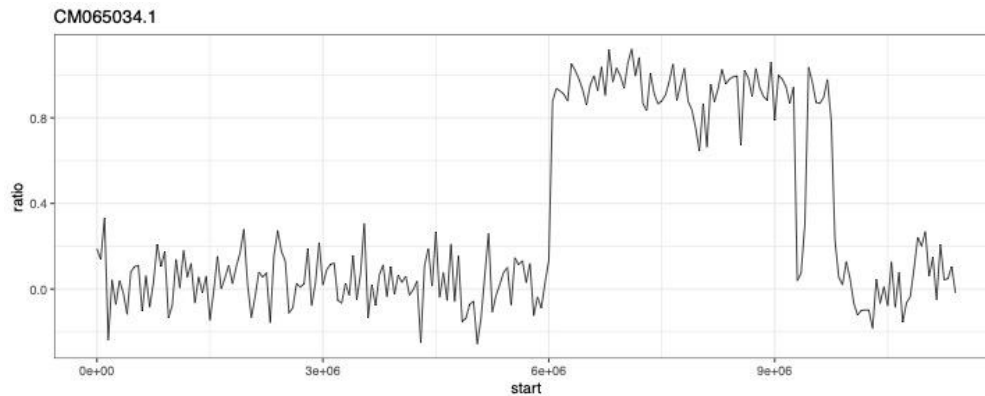


Figure 10. $\text{Log}_2(\text{mutant/control})$ sequencing depth ratio across haplotype CM065034.1

Candidate regions as shown in the three plots, are characterized by the presence of broad, contiguous shifts in the $\text{log}_2(\text{mutant/control})$ coverage ratio that extend over several consecutive genomic windows, typically spanning megabase-scale intervals. These regions appear as clear plateaus deviating from the baseline near zero, with relatively consistent ratio values and well-

defined boundaries separating them from surrounding regions. In contrast to isolated spikes or noisy fluctuations, which are often caused by mapping artifacts or local sequence complexity, true candidate regions show a sustained and coherent shift in coverage, indicating a systematic difference in copy number between mutant and control samples. A total of 33 plots were generated for all 33 haplotypes. Only three were candidates for discovery.

Table 7. The 33 haplotype plots created and the three candidates

Chromosome	Ban Haplotype	ID	Dh haplotype	ID	Ze haplotype	ID
1	Ban01	CM65019.1	Dh01	CM65030.1	Ze01	CM65041.1
2	Ban02	CM65020.1	Dh02	CM65031.1	Ze02	CM65042.1
3	Ban03	CM65021.1	Dh03	CM65032.1	Ze03	CM65043.1
4	Ban04	CM65022.1	Dh04	CM65033.1	Ze04	CM65044.1
5	Ban05	CM65023.1	Dh05	CM65034.1*	Ze05	CM65045.1
6	Ban06	CM65024.1*	Dh06	CM65035.1	Ze06	CM65046.1
7	Ban07	CM65025.1*	Dh07	CM65036.1	Ze07	CM65047.1
8	Ban08	CM65026.1	Dh08	CM65037.1	Ze08	CM65048.1
9	Ban09	CM65027.1	Dh09	CM65038.1	Ze09	CM65049.1
10	Ban10	CM65028.1	Dh10	CM65039.1	Ze10	CM65050.1
11	Ban11	CM65029.1	Dh11	CM65040.1	Ze11	CM65051.1

Regions with an * are candidate haplotypes exhibiting sustained shifts in sequencing depth

To validate candidate regions, sequencing depth was calculated directly from the BAM files using samtools depth. The flanking regions serve as internal controls, allowing coverage within the candidate region to be compared with neighboring genomic regions that are not expected to contain structural variation. The -r option was used to restrict the calculation to a specific genomic interval (e.g., a candidate region or its flanking region), while the -aa option ensures that all positions within the interval are reported, including those with zero coverage. This is important for accurately estimating mean depth across the entire region, as it prevents overestimation that would occur if only covered positions were considered. The output was then summarized using awk to compute the average depth across all bases in the interval, allowing direct comparison of coverage between mutant and control samples in both shifted and flanking regions.

```
# To calculate mean sequencing depth for the candidate (shift) and flanking
regions from haplotype CM065024.1

samtools depth -aa -r CM065024.1:20000000-30000000 SRR11579628.sorted.bam |
awk '{sum+=$3; n++;} \
END{print "CM065024.1 mutant shift", sum/n, n}'
```



```
samtools depth -aa -r CM065024.1:20000000-30000000 SRR11579627.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065024.1 control shift", sum/n, n}'

samtools depth -aa -r CM065024.1:10000000-18000000 SRR11579628.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065024.1 mutant flank", sum/n, n}'

samtools depth -aa -r CM065024.1:10000000-18000000 SRR11579627.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065024.1 control flank", sum/n, n}'

# To extract flanking regions from haplotype CM065025.1

samtools depth -aa -r CM065025.1:4000000-12000000 SRR11579628.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065025.1 mutant shift", sum/n, n}'

samtools depth -aa -r CM065025.1:4000000-12000000 SRR11579627.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065025.1 control shift", sum/n, n}'

samtools depth -aa -r CM065025.1:15000000-23000000 SRR11579628.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065025.1 mutant flank", sum/n, n}'

samtools depth -aa -r CM065025.1:15000000-23000000 SRR11579627.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065025.1 control flank", sum/n, n}'

# To extract flanking regions from haplotype CM065034.1

samtools depth -aa -r CM065034.1:6000000-9000000 SRR11579628.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065034.1 mutant shift", sum/n, n}'

samtools depth -aa -r CM065034.1:6000000-9000000 SRR11579627.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065034.1 control shift", sum/n, n}'

samtools depth -aa -r CM065034.1:1000000-4000000 SRR11579628.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065034.1 mutant flank", sum/n, n}'

samtools depth -aa -r CM065034.1:1000000-4000000 SRR11579627.sorted.bam |
awk '{sum+=$3; n++} \
END{print "CM065034.1 control flank", sum/n, n}'
```

Extraction of the depth information for flanking and candidate regions for each candidate haplotype gave the results summarized Table 8.

Table 8. Depth information for shift and flanking regions

Genome	Depth CM065024 .1-Shift	Depth CM065024 .1- Flanking	Depth CM065025 .1-Shift	Depth CM065025 .1- Flanking	Depth CM065034 .1-Shift	Depth CM065034 .1- Flanking
SRR11579 628 (Mutant)	5.34	1.02	0.32	1.07	0.99	1.06
SRR11579 627 (Control)	8.25	0.93	0.94	0.91	0.05	0.99

The obtained values confirm changes in depth between the flanking regions and the shift regions. For CM065025.1 and CM065034.1 the candidate regions show clear differences in sequencing depth between the mutant and control samples, whereas their corresponding flanking regions show similar coverage. This pattern is consistent with localized structural variation. There is not a drastic change in the shift region between mutant and control for CM065024.1; only between the shift and flanking regions. To further confirm the shifts between mutant and control, he aligned sequencing reads were visualized using** IGV, although other genome visualization software can also be used. For that, we exported the candidate haplotypes using the `samtools view` function and subsequently indexed the extracted regions.

Extracting three candidate shift region to visualize with IGV:

To facilitate visualization, the candidate haplotypes can be extracted as bam files and viewed along the reference genome using any visualization tool. We will use a genome browser called IGV (Integrative Genomics Viewer) which is open source. To extract the regions, the following commands will be used:

```
#Extracting CM065024.1 in mutant and control

samtools view -b SRR11579628.sorted.bam CM065024.1 >
SRR11579628.CM065024.1.bam
samtools index SRR11579628.CM065024.1.bam

samtools view -b SRR11579627.sorted.bam CM065024.1 >
SRR11579627.CM065024.1.bam
samtools index SRR11579627.CM065024.1.bam
```

```
#Extracting CM065025.1 in mutant and control

samtools view -b SRR11579628.sorted.bam CM065025.1 >
SRR11579628.CM065025.1.bam
samtools index SRR11579628.CM065025.1.bam

samtools view -b SRR11579627.sorted.bam CM065025.1 >
SRR11579627.CM065025.1.bam
samtools index SRR11579627.CM065025.1.bam

#Extracting CM065034.1 in mutant and control

samtools view -b SRR11579628.sorted.bam CM065034.1 >
SRR11579628.CM065034.1.bam
samtools index SRR11579628.CM065034.1.bam

samtools view -b SRR11579627.sorted.bam CM065034.1 >
SRR11579627.CM065034.1.bam
samtools index SRR11579627.CM065034.1.bam
```

Visualizations with IGV

The extracted Bam files can be visualized together with the reference genome in IGV which can be downloaded from this site: <https://igv.org/>. For visualizations in IGV, the reference genome GCA_032878665.1_ASM3287866v1_genomic.fna and its corresponding index file, GCA_032878665.1_ASM3287866v1_genomic.fai files were uploaded into IGV along with the BAM files corresponding to each of candidate haplotype. The resulting visualizations for each candidate region are shown in Figure 11, Figure 12, and Figure 13.

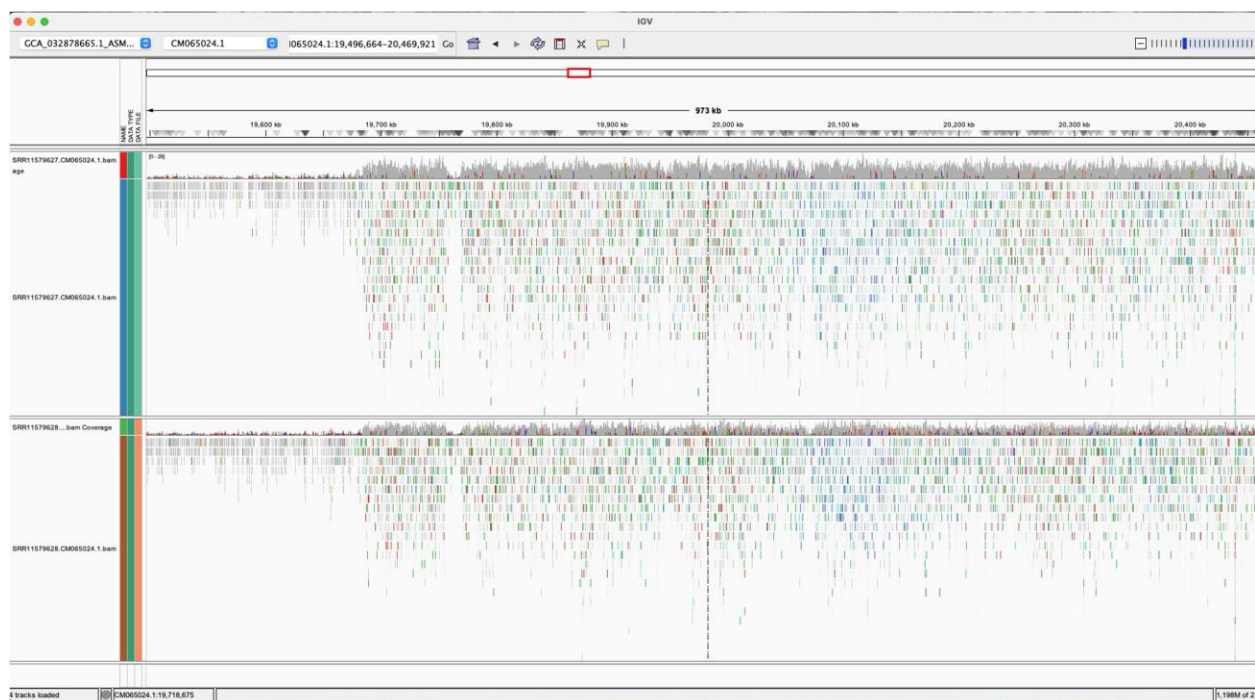


Figure 11. Shift and flanking regions in haplotype CM65024.1

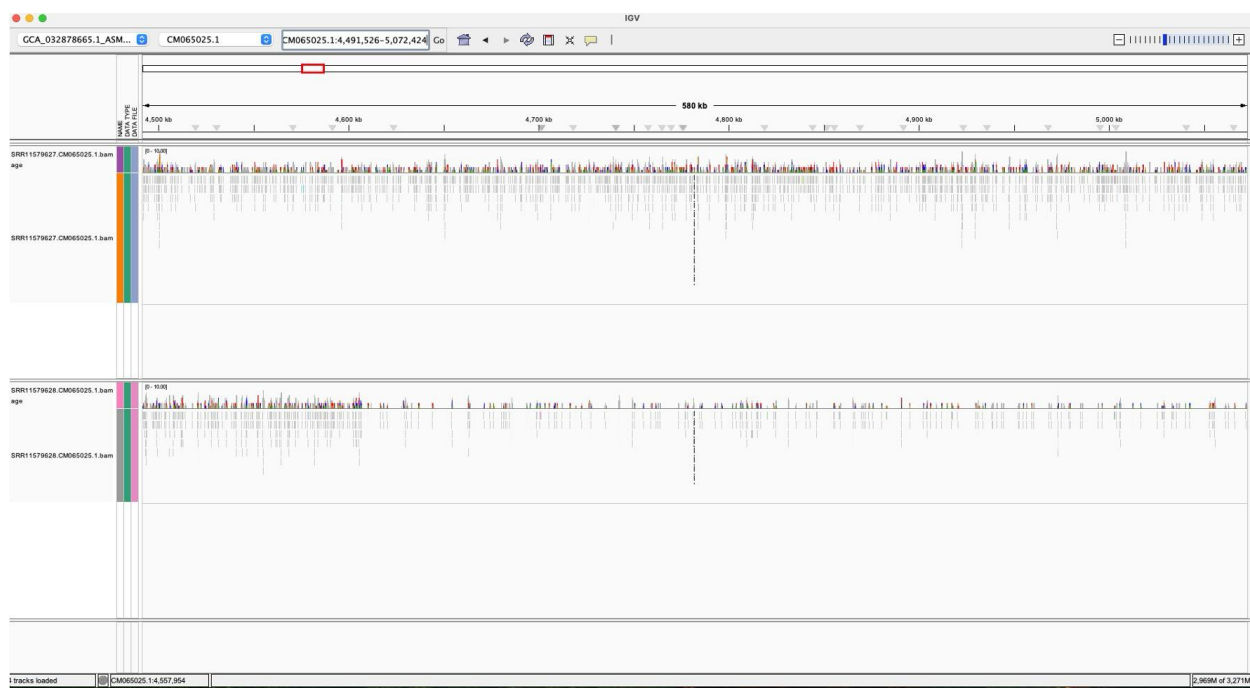


Figure 12. Shift and flanking regions in haplotype CM65025.1

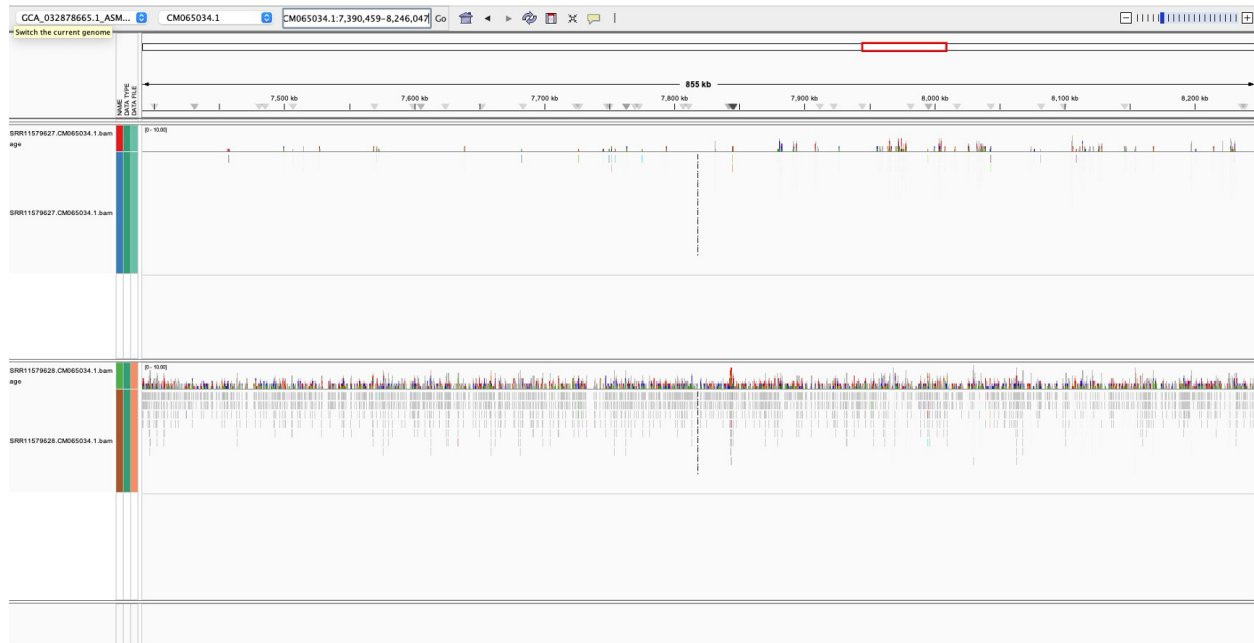


Figure 13. Shift and flanking regions in haplotype CM065034.1

Visualization of the candidate regions in IGV further refines their interpretation by comparing read depth patterns between the mutant and control samples. In the first region, CM065024.1, although a clear shift in coverage is observed between the flanking and candidate interval, the depth profiles of mutant and control are highly similar, suggesting that this shift may reflect local mapping or sequence composition effects rather than a true sample-specific structural change. In contrast, the second region, CM065025.1 shows both a shift relative to the flanking regions and a subtle but consistent reduction in coverage in the mutant compared to the control within the candidate interval, supporting the presence of a mutant-associated dosage change. The third region, CM065034.1 displays a pronounced difference between mutant and control, with a drastic change in coverage within the candidate interval, indicating a strong and likely biologically meaningful structural variation. Based on these observations, CM065025.1 and CM065034.1 were selected for downstream analyses.

Downstream analysis of region CM065034.1

This evaluation is straightforward: the CM065034.1 candidate interval is represented in the reference genome and is supported by reads in the mutant but shows markedly reduced read support in the control. Therefore, CM065034.1 represents a strong candidate region; however, the exact nature and direction of the structural change require additional validation, such as PCR assays targeting the region, local or de novo assembly, or other complementary approaches. Nonetheless, because reads from the mutant align to this interval in the reference genome while reads from the control show substantially lower coverage, we proceed by extracting this region

from the reference genome and annotating the genes it contains. The candidate region can be extracted using samtools with the following command:

```
samtools faidx GCA_032878665.1_ASM3287866v1_genomic.fna \
  CM065034.1:6000000-9000000 \
  > CM065034.1_6_9Mb.ref.fa
```

The `samtools faidx` command extracts the specified genomic interval from the reference genome and writes it to a separate FASTA file for downstream analyses..

With the genomic interval extracted, we could check for the gene annotations within this region. However, because this genome has not been annotated, we will use Augustus, an *ab initio* gene prediction program (Stanke et al., 2006), which identifies likely gene structures directly from genomic DNA sequence. Augustus predicts features such as exons, introns, coding sequences, and complete gene models using species-specific training parameters. It should be noted that the resulting annotations represent computational gene predictions rather than experimentally validated gene annotations.

Augustus requires a species model to guide gene prediction. In this example, we use the rice (*Oryza sativa*) model because it is readily available and performs reasonably well for monocot genomes.

```
augustus \
  --species=rice \
  --protein=on \
  CM065034.1_6_9Mb.ref.fa \
  > CM065034.1_6_9Mb.augustus.with_proteins.gff
```

The number of genes can be counted as follows:

```
grep -P "\tgawk -F'\t' '$3=="gene"' \
  CM065034.1_6_9Mb.augustus.with_proteins.gff > genes.gff
wc -l genes.gff
```

The output of this command indicates that a total of 283 genes were predicted within that the candidate interval. These 283 genes are missing in the same region for the control but are present in the mutant and the reference. Follow up analysis may include BLAST search yes against the

NCBI databases to identify homologous proteins and infer their potential biological functions. The predicted gene annotations will be contained in the file *.gff.

Although CM065034.1 was used here as the working example, the same workflow can be applied to any candidate regions identified from the coverage analysis, such as CM065025.1. For each region, the interval can be extracted from the reference genome, annotated or gene-predicted, and the resulting genes evaluated as potential candidates associated with the observed mutant phenotype. Ultimately, all candidate genes and genomic regions should be validated experimentally using approaches such as qPCR, gene cloning, or gene editing.

Rather than manually examining all predicted genes, we can use a specialized tool to determine whether the candidate region contains resistance (R) genes such as [NLR-Annotator](#). For working with NLR-Annotator we use candidate region CM065025.1 because we knew that this interval contains predicted NLR loci and hence it provides a suitable example of the workflow. The same procedure can be applied to CM065034.1 or any other candidate region.

NLR-Annotator is a Java-based program specifically designed to identify nucleotide-binding leucine-rich repeat (NLR) resistance genes in plant genomes. To run NLR-Annotator, Java must be installed and accessible from the command line. Before running NLR-Annotator, the following files are required:

- NLR-Annotator.jar : main executable JAR file
- mot.txt : motif definition file
- store.txt : motif scoring and configuration file
- Genome FASTA file (*.fa, *.fasta, or *.fna)

Note: The required files can be downloaded from the official NLR-Annotator repository. After downloading, place all files in the same working directory for simplicity.

Java availability can be verified with:

```
java -version
```

If Java is properly installed, version information should be displayed.

Throughout this tutorial, it is assumed that:

1. Java is installed
2. All required NLR-Annotator files are located in the current working directory
3. The genome FASTA file has already been downloaded and is ready for analysis

The analysis can then be executed directly using the following bash script:

```
chmod +x run_nlr.sh
./run_nlr.sh
```

Save the following Bash script as “run_nlr.sh” in the same directory as the FASTA file corresponding to the candidate region to be analyzed.

```
#!/bin/bash
mkdir -p nlr_results

java -Xmx8G -jar NLR-Annotator.jar \
-i CM065025.1_region.ref.fa \
-x mot.txt \
-y store.txt \
-t 4 \
-o nlr_results/nlr_annotator.out.txt \
-g nlr_results/nlr_annotator.gff \
-b nlr_results/nlr_annotator.bed \
-m nlr_results/nlr_motifs.bed \
-a nlr_results/nb_arc_motifs.aln.fasta

java -Xmx8G -jar NLR-Annotator.jar \
-i CM065025.1_region.ref.fa \
-x mot.txt \
-y store.txt \
-t 4 \
-f CM065025.1_region.ref.fa nlr_results/nlr_loci.fasta 0
```

Once the script has been saved, it must be made executable before it can be run:

```
chmod +x run_nlr.sh
```

The analysis can then be executed with:

```
./run_nlr.sh
```


Once the analysis finishes, several output files will be generated in the `nlr_results` directory:

- 1) `nb_arc_motifs.aln.fasta` - Alignment of detected NB-ARC motif regions among candidate NLR loci. Useful for motif-level inspection and comparison.
- 2) `nlr_annotator.bed` - BED-format coordinates of predicted NLR loci. Useful for quick visualization or downstream bedtools analyses.
- 3) `nlr_annotator.gff` - Genome annotation file containing the coordinates of predicted NLR loci and motif annotations. Useful for visualization in genome browsers such as IGV.
- 4) `nlr_annotator.out.txt` - Summary output generated by NLR-Annotator describing the detected motifs and predicted NLR loci.
- 5) `nlr_loci.fasta` - FASTA sequences corresponding to the predicted NLR loci extracted from the input genome region.
- 6) `nlr_motifs.bed` - BED file containing the genomic coordinates of individual detected NLR-associated motifs.

For most downstream analysis, the most informative output files are `nlr_annotator.gff`, `nlr_annotator.bed` and `nlr_loci.fasta`.

A basic inspection of the predicted NLR loci can be performed using the following command:

```
grep -v "^#" nlr_annotator.gff
```

The output is similar to the following:

```
CM065025.1:3000000-13000000 NLR_Annotator NBSLRR 3736850 3738856 . + .
name=CM065025.1:3000000-13000000_nlr1; nlrClass=NBARC-LRR
```

```
CM065025.1:3000000-13000000 NLR_Annotator NBSLRR 7023508 7026498 . - .
name=CM065025.1:3000000-13000000_nlr2; nlrClass=CC-NBARC-LRR
```

```
CM065025.1:3000000-13000000 NLR_Annotator NBSLRR 3710610 3712589 . - .
name=CM065025.1:3000000-13000000_nlr3; nlrClass=NBARC-LRR
```

These results indicate that NLR-Annotator identified three candidate NLR (nucleotide-binding leucine-rich repeat) resistance gene loci within the analyzed genomic interval CM065025.1:3,000,000–13,000,000. Each line represents one predicted NLR locus and includes its genomic coordinates, strand orientation, and predicted structural class. The first candidate locus (`nlr1`) was detected between positions 3,736,850 and 3,738,856 on the positive (+) DNA strand and was classified as an NBARC-LRR type NLR, indicating the presence of both an NB-ARC domain and leucine-rich repeat (LRR) region. The second candidate (`nlr2`) was identified between positions 7,023,508 and 7,026,498 on the negative (-) strand and was classified as CC-NBARC-LRR, suggesting the presence of an N-terminal coiled-coil (CC) domain in addition to the NB-ARC and LRR domains. The third candidate (`nlr3`) was detected between positions 3,710,610 and 3,712,589 on the negative strand and was also classified as NBARC-LRR.

Collectively, these results indicate that the analyzed genomic region contains multiple putative disease resistance genes with characteristic NLR-associated domains.

The `nlr_loci.fasta` file contains the genomic sequences corresponding to the predicted NLR loci identified above. Each sequence can be used as a query in NCBI BLAST to identify homologous genes and infer their potential functions. The resulting annotations are summarized in Table 9.

Table 9. Candidate NLR-associated resistance genes identified within the CM065025.1 candidate region

NLR-Candidate in CM065025 region	NLR Match on NCBI
NBSLRR 3736850 3738856	Musa acuminata AAA Group disease resistance protein RPS2-like (LOC103990280)
NBSLRR 7023508 7026498	Musa acuminata AAA Group disease resistance protein RGA2-like (LOC103991524)
NBSLRR 3710610 3712589	Musa acuminata AAA Group disease resistance protein RPS2-like (LOC135616881)

These results support the NLR-Annotator predictions and indicate that the detected loci likely correspond to bona fide plant disease-resistance genes. The identified annotations include:

- **RPS2-like resistance proteins**, which belong to well-characterized classes of intracellular immune receptors involved in pathogen recognition
- **RGA2-like resistance proteins**, another class of NLR-associated resistance genes commonly involved in plant defense responses

The agreement between the NLR-Annotator predictions and the BLAST annotations provides additional confidence that the candidate genomic region contains genes associated with plant disease resistance.

Although these analyses identify strong candidate genes and genomic regions associated with the mutant phenotype, all bioinformatic predictions should ultimately be validated experimentally using molecular and functional approaches.

Part V - Analysis of microbial datasets

Whether working with long read or short read sequencing data, the quality assessment and pre-processing of the data such as filtering or trimming for microorganisms, is similar to the processes discussed and conducted in previous sections for plants. For short read datasets, such as those generated with Illumina sequencing, both reference-based mapping and de novo genome assembly are commonly used. However, because microbial genomes are generally much smaller and less repetitive than plant genomes, high-quality de novo assemblies can often be generated using short-read data alone.. In this section, we focus on de novo assembly because several assembly tools have been specifically developed and optimized for microbial genomes. When a suitable reference genome is available, reference-based mapping can also be performed using Minimap2, as described earlier in this tutorial.

As an example, we will use an Illumina short-read dataset for *Bacillus velezensis*. The dataset has already been pre-processed as described earlier in the previous sections, and its taxonomic identity has been confirmed using KrakenUniq (Wood and Salzberg, 2014). KrakenUniq is a k-mer-based taxonomic classification tool that rapidly assigns sequencing reads to known taxa by comparing them with a reference database of microbial genomes. It can classify reads to the species, genus, or higher taxonomic levels, allowing rapid confirmation of the organism present in a sequencing dataset before genome assembly. In addition to taxonomic identification, KrakenUniq is widely used for quality control and contamination detection, enabling the identification of host DNA, environmental contaminants, or unintended microbial species that may compromise downstream analyses such as genome assembly and functional annotation. An example KrakenUniq command for taxonomic classification of paired-end Illumina FASTQ files is shown below:

```
krakenuniq --db /path/to/krakenuniq_db \  
  --threads 16 \  
  --paired bacillus_1.fastq bacillus_2.fastq \  
  --report-file bacillus.report.tsv \  
  > bacillus.output.txt
```

Where the `--db /path/to/krakenuniq_db` option specifies the path to the KrakenUniq database used for classification. The `--threads 16` option defines the number of CPU threads to use, allowing parallel processing and reducing runtime. The `--paired` option indicates that the input consists of paired-end reads, provided as `bacillus_1.fastq` and `bacillus_2.fastq`. The `--report-file bacillus.report.tsv` option generates a summary report of taxonomic classifications, including read counts and unique k-mer counts for each taxon. The standard output is redirected to `bacillus.output.txt`, which contains the per-read classification results.

Once the dataset has passed quality assessment using FastQC and Fastp, and KrakenUniq confirms the expected organism with no evidence of significant contamination, the reads are ready for downstream analyses such as de novo genome assembly.

De-novo assembly of short reads (Illumina - small, bacteria, fungus)

De-novo assembly for microbial data can be done using SPAdes (Bankevich et al., 2012) using the following commands:

```
#careful option:

spades.py --careful -1 bacillus_1.fastq -2 bacillus_2.fastq \
  -o bacillus_spades_output \
  -t 16 -m 64

#isolate option:

spades.py --isolate -1 bacillus_1.fastq -2 bacillus_2.fastq \
  -o bacillus_spades_output \
  -t 16 -m 64
```

The command `spades.py` performs de novo genome assembly from sequencing reads. The `-1 bacillus_1.fastq` and `-2 bacillus_2.fastq` options specify the paired-end forward and reverse read files, respectively. The `-o bacillus_spades_output` option defines the output directory where all assembly results will be written. The `-t 16` option sets the number of CPU threads to use, allowing the assembly to run in parallel, while `-m 64` sets the maximum amount of memory (in gigabytes) available to SPAdes. The optional `--isolate` flag indicates that the sequencing data originate from a single microbial isolate, enabling assembly optimizations for isolate genomes. Alternatively, the `--careful` reduces mismatches and insertion/deletion (indel) errors in the final assembly, improving assembly accuracy..

Both `--careful` and `--isolate` options are suitable for sequencing datasets originating from a single microbial isolate. The `--isolate` option optimizes the assembly for isolate genomes, whereas `--careful` focuses on reducing mismatches and small insertion/deletion errors in the final assembly. These options are mutually exclusive and should not be used simultaneously. The primary output of SPAdes is the assembled genome in FASTA format. The primary output of SPAdes is the assembled genome in FASTA format (contigs.fasta), together with additional files containing assembly graphs and assembly statistics.

The same QUAST and BUSCO workflow described earlier can be used to evaluate microbial genome assemblies. The main difference is that an appropriate microbial lineage dataset should

be selected for BUSCO, such as `bacillales_odb12` for members of the order Bacillales or `bacteria_odb12` for a broader bacterial assessment.

```
busco -i bacillus_spades_output/contigs.fasta \
-l ./busco_downloads/bacillales_odb12/ \
-o bacillus_busco_bacillales \
-m genome \
--offline --force
```

The command `busco` assesses the genome assembly completeness by searching for sets of conserved single-copy orthologs. The `-i bacillus_spades_output/contigs.fasta` option specifies the input assembly file generated by SPAdes. The `-l` option specifies the BUSCO lineage dataset used for comparison, such as `bacillales_odb12` for a lineage-specific assessment or `bacteria_odb12` for a broader evaluation. The `-o` option defines the output directory for BUSCO results, while `-m genome` indicates that the input is a genome assembly. The `--offline` flag ensures that BUSCO uses locally downloaded lineage datasets, and `--force` allows overwriting of existing output directories if present.

Once the genome assembly has been generated and its quality verified, we can proceed to mutation discovery to identify genomic differences between mutant and wild-type strains.

Discovering mutations induced with gamma irradiation associated to a specific phenotype in microorganisms

We will work with a *Bacillus* sp. dataset comprising 3 gamma-irradiated mutants and a wild type (non-irradiated control). These genomes were de novo assembled with SPAdes in the previous section.

Before any comparative downstream analysis, contigs smaller than 1,000 bp are removed to improve the reliability of downstream analysis by retaining higher confidence assemblies. Contigs longer than 1,000 bp can be selected using the following Seqkit command.

```
seqkit seq -m 1000 assembly.fasta > assembly_min1000.fasta
```

This command scans all sequences in `assembly.fasta` and retains only those that are at least 1,000 bp long (the `-m 1000` flag sets the minimum length threshold). The output of filtered sequences is then redirected (`>`) into a new file called `assembly_min1000.fasta`. Filtered files can be found in this link in order to proceed with the analysis described below: [Mutant24](#) [Mutant34](#) [Mutant58](#) [WT](#).

The next step is to compare each **mutant genome assembly** with the **wild-type (WT) assembly** using MUMmer. Mummer performs a pairwise alignment of the *Bacillus* genomes and identifies the regions where they differ.

```
dnadiff -p WT_vs_Mutant24 ./WT.fasta
./Mutant24.fasta
dnadiff -p WT_vs_Mutant34 ./WT.fasta
./Mutant34.fasta
dnadiff -p WT_vs_Mutant58 ./WT.fasta
./Mutant58.fasta
```

The `dnadiff` is the command within MUMmer package that is needed to performs pairwise whole-genome comparisons between closely related genome assemblies. In this example, each mutant assembly is compared with the wild-type assembly..

The commands generates several output files: `*.report`, `*.delta`, `*.1delta`, `*.mdelta`, `*.1coords`, `*.mcoords`, `*.rdiff`, and `*.qdiff`.

Alignment summaries contained in the `.coords` files are useful for assessing overall genome collinearity, whereas breakpoint information reported in the `.qdiff` and `.rdiff` files is more informative for identifying insertions, deletions, and structural differences between assemblies. The most relevant files for comparing mutants and WT genomes are the `*.report` file, which summarizes overall alignment, the `*.lcoords` file, which contains one-to-one block alignments, the `*.mcoord` file which reports multiple alignments in highly fragmented assemblies, and the `*.rdiff` and `*.qdiff` files, describe candidate structural differences between the assemblies.

For mutation discovery, the most informative results are obtained from the `*.qdiff` file, which reports candidate structural differences between the mutant and wild-type assemblies. The location and type of changes identified in each mutant compared to the WT are summarized in Table 10.

Table 10. Results for the location and type of changes that happened in the mutants compared to the WT

WT contig name	Type of event	Start position in WT	End position in WT	Change relative to WT (bp)	Change in mutant (bp)	Net difference (bp)
WT_Mutant24						
NODE_2_length_1021494_cov_180.018797	GAP	70796	70784	-11	45	-56
NODE_4_length_272161_cov_168.294172	GAP	143491	143461	-29	157	-186
NODE_4_length_272161_cov_168.294172	GAP	154285	153830	-454	-733	279
NODE_4_length_272161_cov_168.294172	GAP	156803	156245	-557	-683	126
NODE_8_length_4422_cov_1551.432645	GAP	1276	1375	100	100	0
WT_Mutant34						
NODE_4_length_272161_cov_119.210618	GAP	115917	115359	-557	-683	126
NODE_4_length_272161_cov_119.210618	GAP	118332	117877	-454	-733	279
NODE_4_length_272161_cov_119.210618	GAP	128701	128671	-29	157	-186
NODE_8_length_4422_cov_1047.648041	GAP	1276	1375	100	100	0
WT_Mutant58						
NODE_4_length_271774_cov_146.812740	GAP	128314	128229	-84	102	-186

NODE_8_length_4422_cov_1146.366566	GAP	1276	1375	100	100	0
------------------------------------	-----	------	------	-----	-----	---

Because all mutants exhibit the same phenotype, mutations shared among all mutants are most likely to be the strongest candidates for the causative mutation. In this dataset, a common 186 bp GAP event on NODE_4 (~128 kb) represents the primary candidate region.

The NODE_8 event occurs on a short, high-coverage contig corresponding to a repetitive genomic region. Without annotation, its exact identity is unclear, as such regions are prone to assembly ambiguities; therefore, this event is considered a low-confidence candidate.

If SNPs are also detected, the same prioritization strategy should be applied by identifying variants that are consistently present across all mutants but absent from the wild type.

To determine which genes are affected within the candidate regions, the genomes can be annotated using Prokka (Seemann, 2014); a rapid annotation pipeline for prokaryotic genomes.

```
prokka --outdir WT_annot --prefix WT ./WT.fasta
prokka --outdir Mutant24_annot --prefix Mutant24 ./Mutant24.fasta
prokka --outdir Mutant34_annot --prefix Mutant34 ./Mutant34.fasta
prokka --outdir Mutant58_annot --prefix Mutant58 ./Mutant58.fasta
```

The `--outdir` specifies the output directory for annotation results and `--prefix` defines the filename prefix for all generated files. Prokka generates multiple output files, including nucleotide sequences, protein sequences, and annotation files. Among these, the `.gff` file is the primary annotation file containing gene coordinates and functional annotations used for downstream analysis.

With the WT annotation file we can confirm the candidate regions, using the following script:

```
awk '$1=="NODE_8_length_4422_cov_1163.700904" && $4<=1800 && $5>=800' WT.gff
```

This command uses `awk` to extract specific annotation lines from a GFF file by applying filtering conditions: it selects only those entries where the first column (\$1), which corresponds to the contig name, matches `NODE_8_length_4422_cov_1163.700904`, and where the feature coordinates overlap the region between positions 800 and 1800. The overlap is determined by checking that the feature start position (\$4) is less than or equal to 1800 and the feature end position (\$5) is greater than or equal to 800, meaning any feature that falls fully or partially

within this interval is included. In practical terms, this command retrieves all annotated genomic features located on NODE_8 that intersect the region of interest, allowing you to identify genes or elements near a mutation.

This script checks NODE_8 ~1,276–1,375 and the output is summarized in Table 11.

Table 11. Results for Node8

Contig	Feature	Start	End	Product
NODE_8	rRNA	1	1331	16S ribosomal RNA
NODE_8	rRNA	1455	4384	23S ribosomal RNA

NODE_8 corresponds to an rRNA operon region (16S–23S), which is highly repetitive and prone to assembly artifacts. Therefore, differences observed in this region are considered low-confidence and unlikely to represent true biological mutations. NODE_8 is an rRNA contig, and the observed GAP lies in a repeat-rich region, so the observed GAP is not prioritized as a candidate mutation..

To evaluate the candidate region on NODE_4, (~128,314–128,470), we can use the following script:

```
awk '$1=="NODE_4_length_271960_cov_146.200294" && $4<=129000 && $5>=127500' \
  \ WT.gff
```

The output is summarized in Table 12.

Table 12. Results for Node 4

Contig	Feature	Start	End	Strand	Product	Function
NODE_4	yflT	127382	127729	negative	General stress protein 17M	Stress protein
NODE_4	yfmB	127900	128271	positive	Putative protein YfmB	Unknown
NODE_4	yfmJ	128586	129605	positive	NADP-dependent oxidoreductase	Metabolism, redox balance, stress response

The candidate GAP is located at approximately positions 128,314–128,470. The flanking annotated genes are *yfmB* and *yfmJ*, whereas *yflT* is located upstream of the candidate interval.. Because MUMmer reports alignment breakpoints and not exactly biological mutation

boundaries, the reported coordinates should be regarded as approximate and may vary slightly between comparisons.. Therefore, the candidate mutation is most likely located within the intergenic region between *yfmB* and *yfmJ* rather than within a protein-coding sequence. Since neither coding sequence appears to be directly disrupted, the mutation is more likely to affect regulatory elements controlling gene expression than protein function.

REFERENCES

- Andrews, S. 2010. FastQC: a quality control tool for high throughput sequence data. Available online at: [FastQC](#)
- Bankevich, A., Nurk, S., Antipov, D., Gurevich, A.A., Dvorkin, M., Kulikov, A.S., Lesin, V.M., Nikolenko, S.I., Pham, S., Prjibelski, A.D., Pyshkin, A.V., Sirotkin, A.V., Vyahhi, N., Tesler, G., Alekseyev, M.A. & Pevzner, P.A. 2012. SPAdes: a new genome assembly algorithm and its applications to single-cell sequencing. *Journal of Computational Biology*, 19(5): 455–477. <https://doi.org/10.1089/cmb.2012.0021>
- Chen, S., Zhou, Y., Chen, Y. & Gu, J. 2018. fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics*, 34(17): i884–i890. <https://doi.org/10.1093/bioinformatics/bty560>
- Cheng, H., Concepcion, G.T., Feng, X., Zhang, H. & Li, H. 2021. Haplotype-resolved de novo assembly using phased assembly graphs with hifiasm. *Nature Methods*, 18(2): 170–175. <https://doi.org/10.1038/s41592-020-01056-5>
- De Coster, W. & Rademakers, R. 2023. NanoPack2: population-scale evaluation of long-read sequencing data. *Bioinformatics*, 39(5): btad311. <https://doi.org/10.1093/bioinformatics/btad311>
- Hon, T., Mars, K., Young, G. et al. 2020. Highly accurate long-read HiFi sequencing data for five complex genomes. *Scientific Data*, 7: 399. <https://doi.org/10.1038/s41597-020-00743-4>
- Pucker, B., Busche, M., Viehöver, P., Weisshaar, B. & Stracke, R. 2019. Genome assemblies of *Musa acuminata* Dwarf Cavendish. Bielefeld University.
- Robinson, J.T., Thorvaldsdóttir, H., Turner, D. & Mesirov, J.P. 2023. igv.js: an embeddable JavaScript implementation of the Integrative Genomics Viewer (IGV). *Bioinformatics*, 39(1): btac830. <https://doi.org/10.1093/bioinformatics/btac830>
- Robinson, J.T., Thorvaldsdóttir, H., Wenger, A.M., Zehir, A. & Mesirov, J.P. 2017. Variant review with the Integrative Genomics Viewer (IGV). *Cancer Research*, 77(21): e31–e34. <https://doi.org/10.1158/0008-5472.CAN-17-0337>
- Robinson, J.T., Thorvaldsdóttir, H., Winckler, W., Guttman, M., Lander, E.S., Getz, G. & Mesirov, J.P. 2011. Integrative Genomics Viewer. *Nature Biotechnology*, 29(1): 24–26. <https://doi.org/10.1038/nbt.1754>
- Seemann, T. 2014. Prokka: rapid prokaryotic genome annotation. *Bioinformatics*, 30(14): 2068–2069. <https://doi.org/10.1093/bioinformatics/btu153>
- Stanke M, Keller O, Gunduz I, Hayes A, Waack S, Morgenstern B. AUGUSTUS: ab initio prediction of alternative transcripts. *Nucleic Acids Res.* 2006 Jul 1;34(Web Server issue):W435-9. doi: 10.1093/nar/gkl200. PMID: 16845043; PMCID: PMC1538822.
- Thorvaldsdóttir, H., Robinson, J.T. & Mesirov, J.P. 2013. Integrative Genomics Viewer (IGV): high-performance genomics data visualization and exploration. *Briefings in Bioinformatics*, 14(2): 178–192. <https://doi.org/10.1093/bib/bbs017>
- Wick, R.R. 2018. Filtlong: a tool for filtering long reads by quality. Available online at: [Filtlong GitHub repository](#)
- Wood, D.E. & Salzberg, S.L. 2014. Kraken: ultrafast metagenomic sequence classification using exact alignments. *Genome Biology*, 15(3): R46. <https://doi.org/10.1186/gb-2014-15-3-r46>